Video Mixer v6.0

LogiCORE IP Product Guide

Vivado Design Suite

PG243 (v6.0) November 20, 2025

Table of Contents

Introduction

The AMD LogiCORE™ IP Video Mixer core provides a flexible video processing block for alpha blending and compositing multiple video and/or graphics layers. Support for up to seventeen layers (one main layer and sixteen overlay layers), with an optional logo layer, using a combination of video inputs from either memory or streaming video cores (through AXI4-Stream interfaces) is provided. The core is programmable through a comprehensive register interface to control frame size, background color, layer position, and the AXI4-Lite interface. A comprehensive set of interrupt status bits is provided for processor monitoring.

Features

- Supports (per pixel) alpha-blending of seventeen video/graphics and logo layers video/graphics
- Optional logo (in block RAM) layer with color transparency support
- Layers can either be memory mapped AXI4 interface or AXI4-Stream
- Provides programmable background color
- Provides programmable layer position and size
- Provides upscaling of layers by 1x, 2x, or 4x
- Optional built-in color space conversion and chroma re-sampling
- Supports RGB, YUV 444, YUV 422, YUV 420
- Supports memory video format in raster and tile mode
- Supports 8, 10, 12, and 16 bits per color component input and output on stream interface, 8, 10, and 12 bits per color component on memory interface
- Supports semi-planar memory formats next to packed memory formats
- Supports spatial resolutions from 64 × 64 up to 8,192 × 4,320
- Supports 8K60 in all supported device families.

  Note: Performance on low power devices might be lower.
- Supports Programmable CSC coefficients to support various calorimetry like BT601, BT709, and BT2020

- Supports 1, 2, 4, or 8 samples per clock
- Supports programmable CSC coefficient registers

IP Facts

AMD LogiCORE™ IP Facts Table	
Core Specifics	
Supported Device Family[1]	AMD Versal™ Adaptive SoC, AMD UltraScale+™ Families, AMD UltraScale™ Architecture, AMD Zynq™ 7000 SoC, 7 series FPGAs
Supported User Interfaces	AXI4-Lite Master, AXI4-Lite, AXI4-Stream[2]
Resources	Performance and Resource Use web page
Provided with Core	
Design Files	Not Provided
Example Design	Yes
Test Bench	Not Provided
Constraints File	Xilinx Design Constraints (XDC)
Simulation Model	Encrypted RTL
Supported S/W Driver[2]	Standalone DRM/KMS
Tested Design Flows[3]	
Design Entry	AMD Vivado™ Design Suite
Simulation	For supported simulators, see the *Vivado Design Suite User Guide: Release Notes, Installation, and Licensing* (UG973).
Synthesis	Vivado Synthesis
Support	
Release Notes and Known Issues	Master Answer Record: AR 66753
All Vivado IP Change Logs	Master Vivado IP Change Logs: 72775
Support web page	

Notes:

1. For a complete list of supported devices, see the AMD Vivado™ IP catalog.
2. Video protocol as defined in the *AXI4-Stream Video IP and System Design Guide* (UG934).
3. Standalone driver details can be found in <install_directory>/Vitis/<release>/data/embeddedsw/doc/xilinx_drivers_api_toc.htm. Linux OS and driver support information is available from the Wiki page.
4. For the supported versions of third-party tools, see the *Vivado Design Suite User Guide: Release Notes, Installation, and Licensing* (UG973).

Overview

The Video Mixer IP produces one single output video stream from multiple external video and/or graphics sources. Sources can be dynamically positioned, scaled, and combined using alpha-blending.

Navigating Content by Design Process

AMD Adaptive Computing documentation is organized around a set of standard design processes to help you find relevant content for your current development task. You can access the AMD Versal™ adaptive SoC design processes on the Design Hubs page. You can also use the Design Flow Assistant to better understand the design flows and find content that is specific to your intended design needs. This document covers the following design processes:

- **Hardware, IP, and Platform Development:** Creating the PL IP blocks for the hardware platform, creating PL kernels, functional simulation, and evaluating the AMD Vivado™ timing, resource use, and power closure. Also involves developing the hardware platform for system integration. Topics in this document that apply to this design process include:
  - Port Descriptions
  - Register Space
  - Clocking
  - Resets
  - Customizing and Generating the Core
  - Chapter 6: Example Design

Feature Summary

The Video Mixer is a highly configurable IP core that supports blending of up to seventeen video and/or graphics layers plus an additional logo layer into one single output video stream. All layers except the master layer can either be a memory mapped AXI4 interface or AXI4-Stream based. Alpha-blending (global or per pixel) and scaling is supported per layer. Finally, built-in color space conversion between RGB and YUV 4:4:4 and chroma re-sampling between YUV 4:4:4, YUV 4:2:2, and YUV 4:2:0 is optionally available.

The mixer provides an optional logo layer. It blends a logo that is stored in block RAM on the topmost layer. A programmable color key can be used to make part of the logo transparent. Also, alpha-blending (global or per pixel) can be used for logo transparency.

Alpha-blending is the convex combination of two image layers allowing for transparency. Each layer in the mixer has a fixed Z-plane order; or conceptually, each layer resides closer to the observer having a different depth. Thus, the image and the image directly "over" it are blended. The order and amount of blending is programmable in real-time. This can either be done through a global alpha, that is, every pixel uses the same alpha value, or through a per pixel alpha value in case either the RGBA8, BGRA8, or YUVA8 memory video format is selected. Per pixel alpha also supports RGBA and YUVA444 streaming video formats.

Scaling by means of pixel and line repeat is supported, and provides scaling of layers by 1x, 2x, or 4x. This feature might be used to save on memory bandwidth used by a layer, that is, a memory layer can read in a 1,920 × 1,080 frame buffer while scaling it up on-the-fly to a 3,840 × 2,160 resolution.

The Video Mixer supports parallel processing of multiple pixels per clock cycle allowing support for resolutions beyond 1080p60 with up to three color components, each of 8, 10, 12, or 16 bits.

Applications

Applications range from broadcast and consumer, automotive, medical, and industrial imaging and can include the following:

- Video surveillance
- Machine vision
- Video conferencing
- Set-top box displays

Licensing and Ordering

This AMD LogiCORE™ IP module is provided at no additional cost with the AMD Vivado™ Design Suite under the terms of the End User License.

Information about other AMD LogiCORE™ IP modules is available at the Intellectual Property page. For information about pricing and availability of other AMD LogiCORE IP modules and tools, contact your local sales representative.

For more information about this core, visit the Video Mixer product web page.

Chapter 3

Product Specification

Standards

The Video Mixer is compliant with the AXI4-Stream Video Protocol, AXI4-Lite interconnect and memory mapped AXI4 interface standards. For additional information, see the Video IP: AXI Feature Adoption section of the *AXI4-Stream Video IP and System Design Guide* (UG934).

Performance

The following sections detail the performance characteristics of the Video Mixer.

Maximum Frequencies

The following are typical clock frequencies for the target devices:

- AMD Virtex™ 7 and AMD Virtex™ UltraScale™ devices with –2 speed grade or higher: 300 MHz
- AMD Kintex™ 7 and AMD Kintex™ UltraScale™ devices with –2 speed grade or higher: 300 MHz
- AMD Artix™ 7 devices with –2 speed grade or higher: 150 MHz
- AMD UltraScale+™ devices with -1 speed grade or higher: 300 MHz
- AMD Versal™ Adaptive SoC devices with -1 Speed grade or higher: 300 MHz

The maximum achievable clock frequency can vary. The maximum achievable clock frequency and all resource counts can be affected by other tool options, additional logic in the device, using a different version of AMD tools, and other factors.

Throughput

The Video Mixer supports bi-directional data throttling between its AXI4-Stream slave and master interfaces. If the slave side data source is not providing valid data samples (`s_axis_video_tvalid` is not asserted), the core cannot produce valid output samples after its internal buffers are depleted. Similarly, if the master side interface is not ready to accept valid data samples (`m_axis_video_tready` is not asserted) the core cannot accept valid input samples after its buffers become full.

If the master interface is able to provide valid samples (`s_axis_video_tvalid` is High) and the slave interface is ready to accept valid samples (`m_axis_video_tready` is High), typically the core can process and produce one, two, four, and eight pixels specified by Samples Per Clock in the AMD Vivado™ Integrated Design Environment (IDE) per `ap_clk` cycle.

However, at the end of each scan line and frame the core flushes internal pipelines for several clock cycles, during which the `s_axis_video_tready` is deasserted signaling that the core is not ready to process samples.

When the Video Mixer is processing timed streaming video (which is typical for most video sources), the flushing periods coincide with the blanking periods and therefore do not reduce the throughput of the system.

When operating on a streaming video source (that is, not frame buffered data), the Video Mixer must operate minimally at the burst data rate. For example, 148.5 MHz for a 1080p60 video source for a one sample per clock configuration of the IP. For a 4K 60 fps video source, the core must operate at 297 MHz for a two sample per clock configuration, or 148.5 MHz for a four sample per clock configuration on slower devices such as AMD Artix™ 7.

Resource Utilization

For full details about performance and resource utilization, visit the Performance and Resource Utilization web page.

Port Descriptions

The Video Mixer uses industry standard control and data interfaces to connect to other system components. The following sections describe the various interfaces available with the core. The following figure illustrates a Video Mixer I/O diagram. In this configuration, the IP has 10 AXI interfaces:

- AXI4-Lite control interface (`s_axi_CTRL`)

- AXI4-Stream streaming video output (`m_axis_video`)
- AXI4-Stream streaming video input (`s_axis_video`, `s_axis_video1`, etc.)
- Memory mapped AXI4 interface (`m_axi_mm_video4`, `m_axi_mm_video5`, etc.).

Figure 1: **Video Mixer I/O Diagram**

The interfaces change depending on the number of layers and layer type, that is, streaming or memory based. For example, the following figure shows a minimum configuration of the Video Mixer IP with only one layer and a logo layer.

Figure 2: **Video Mixer I/O Diagram Single Layer**

Common Interface Signals

The following table summarizes the signals which are either shared by, or not part of the dedicated AXI4-Stream, memory mapped AXI4 data, or AXI4-Lite control interfaces.

Table 1: **Common Interface Signals**

Signal Name	I/O	Width	Description
ap_clk	I	1	Video core clock
ap_rst_n	I	1	Video core active-Low reset
interrupt	O	1	Interrupt Request Pin

The `ap_clk` and `ap_rst_n` signals are shared between the core, the AXI4-Stream, memory mapped AXI4 data interfaces, and the AXI4-Lite control interface.

ap_clk

The AXI4-Stream, memory mapped AXI4, and AXI4-Lite interfaces must be synchronous to the core clock signal `ap_clk`. All AXI4-Stream, memory mapped AXI4 interface input signals and AXI4-Lite control interface input signals are sampled on the rising edge of `ap_clk`. All AXI4-Stream output signal changes occur after the rising edge of `ap_clk`.

ap_rst_n

The `ap_rst_n` pin is an active-Low, synchronous reset input pertaining to both AXI4-Lite, AXI4-Stream, and memory mapped AXI4 interfaces. When `ap_rst_n` is set to 0, the core resets at the next rising edge of `ap_clk`.

interrupt

The interrupt status output bus can be integrated with an external interrupt controller that has independent interrupt enable/mask, interrupt clear, and interrupt status registers that allows for interrupt aggregation to the system processor.

AXI4-Stream Video Interface

The Video Mixer in any configuration has AXI4-Stream video input and output interfaces named `s_axis_video` and `m_axis_video`, respectively. Per additional streaming layer, there are an additional AXI4-Stream video input named `s_axis_videoi` with `i` representing the layer number minus 1. All video streaming interfaces follow the interface specification as defined in the *AXI4-Stream Video IP and System Design Guide* (UG934). The video AXI4-Stream interface can be single, dual, quad, or octa pixels per clock and can support 8, 10, 12, or 16 bits per component. The streaming interface configuration (samples per clock and bits per component) is chosen at IP level and applies to all instances of the AXI4-Stream interface.

The following tables Dual Pixels Per Clock, 10 Bits Per Component Mapping for RGB through Dual Pixels Per Clock, 10 Bits Per Component Mapping for YUVA 4:4:4 explain the pixel mapping of an AXI4-Stream interface with two pixels per clock and 10 bits per component configuration for all supported color formats. Given that the Video Mixer always requires a hardware configuration for at least three component video, the AXI4-Stream Subset Converter is needed to communicate with other IPs of two or one component video interface in YUV 4:2:2, 4:2:0, or Luma Only.

Table 2: **Dual Pixels Per Clock, 10 Bits Per Component Mapping for RGB**

63:60	59:50	49:40	39:30	29:20	19:10	9:0
Zero padding	R1	B1	G1	R0	B0	G0

Table 3: **Dual Pixels Per Clock, 10 Bits Per Component Mapping for YUV 4:4:4**

63:60	59:50	49:40	39:30	29:20	19:10	9:0
Zero padding	V1	U1	Y1	V0	U0	Y0

Table 4: **Dual Pixels Per Clock, 10 Bits Per Component Mapping for YUV 4:2:2**

63:60	59:50	49:40	39:30	29:20	19:10	9:0
Zero padding	Zero padding	Zero padding	V0	Y1	U0	Y0

Table 5: **Dual Pixels Per Clock, 10 Bits Per Component Mapping for RGBA**

79:70	69:60	59:50	49:40	39:30	29:20	19:10	9:0
A1	R1	B1	G1	A0	R0	B0	G0

Table 6: **Dual Pixels Per Clock, 10 Bits Per Component Mapping for YUVA 4:4:4**

79:70	69:60	59:50	49:40	39:30	29:20	19:10	9:0
A1	V1	U1	Y1	A0	V0	U0	Y0

The following table shows the interface signals for input and output AXI4-Stream video streaming interfaces.

Table 7: **AXI4-Stream Interface Signals**

Signal Name	I/O	Width	Description
s_axis_tdata	I	floor(((3[1(1)] × bits_per_component × pixels_per_clock) + 7) / 8) × 8	Input Data
s_axis_tready	O	1	Input Ready

Table 7: **AXI4-Stream Interface Signals** *(cont'd)*

Signal Name	I/O	Width	Description
s_axis_tvalid	O	1	Input Valid
s_axis_tdest	I	1	Input Data Routing Identifier
s_axis_tkeep	I	(s_axis_video_tdata width) / 8	Input byte qualifier that indicates whether the content of the associated byte of TDATA is processed as part of the data stream.
s_axis_tlast	I	1	Input End of Line
s_axis_tstrb	I	(s_axis_video_tdata width) / 8	Input byte qualifier that indicates whether the content of the associated byte of TDATA is processed as a data byte or a position byte.
s_axis_tuser	I	1	Input Start of Frame
m_axis_tdata	O	floor(((3 × bits_per_component × pixels_per_clock) + 7) / 8) × 8	Output Data
m_axis_tdest	O	1	Output Data Routing Identifier
m_axis_tid	O	1	Output Data Stream Identifier
m_axis_tkeep	O	(m_axis_video_tdata width) / 8	Output byte qualifier that indicates whether the content of the associated byte of TDATA is processed as part of the data stream.
m_axis_tlast	O	1	Output End of Line
m_axis_tready	I	1	Output Ready
m_axis_tstrb	O	(m_axis_video_tdata width) / 8	Output byte qualifier that indicates whether the content of the associated byte of TDATA is processed as a data byte or a position byte.
m_axis_tuser	O	1	Output Start of Frame
m_axis_tvalid	O	1	Output Valid

Notes:

1. For RGBA and YUVA444, the video data consists of four components.

All video streaming interfaces run at the IP core clock speed, `ap_clk`.

Memory Mapped AXI4 Interface

Per memory layer, there is a memory mapped AXI4 interface named `m_axi_mm_video i` with `i` representing the layer number minus 1. The memory mapped AXI4 interface runs on the `ap_clk` clock domain. The signals follow the specification as defined in the *Vivado Design Suite: AXI Reference Guide* (UG1037). The following table shows the pixel formats with raster scan order in memory supported by the Video Mixer.

Table 8: **Pixel Formats**

Video Format	Description	Bits per Component	Bytes per Pixel
RGBX8	packed RGB	8	4 bytes per pixel
BGRX8	packed BGR	8	4 bytes per pixel
YUVX8	packed YUV 4:4:4	8	4 bytes per pixel

Table 8: **Pixel Formats** *(cont'd)*

Video Format	Description	Bits per Component	Bytes per Pixel
YUYV8	packed YUV 4:2:2	8	2 bytes per pixel
UYVY8	packed YUV 4:2:2	8	2 bytes per pixel
RGBA8	packed RGB with alpha	8	4 bytes per pixel
BGRA8	packed BGR with alpha	8	4 bytes per pixel
YUVA8	packed YUV 4:4:4	8	4 bytes per pixel
RGBX10	packed RGB	10	4 bytes per pixel
YUVX10	packed YUV 4:4:4	10	4 bytes per pixel
RGB565	packed RGB	5 bits per R component 6 bits per G component 5 bits per B component	2 bytes per pixel
BGR8	packed BGR	8	3 bytes per pixel
Y_UV8	semi-planar YUV 4:2:2	8	1 byte per pixel per plane
Y_UV8_420	semi-planar YUV 4:2:0	8	1 byte per pixel per plane
RGB8	packed RGB	8	3 bytes per pixel
YUV8	packed YUV 4:4:4	8	3 bytes per pixel
Y_UV10	semi-planar YUV 4:2:2	10	4 bytes per 3 pixels per plane
Y_UV10_420	semi-planar YUV 4:2:0	10	4 bytes per 3 pixels per plane
Y8	packed luma only	8	1 byte per pixel
Y10	packed luma only	10	4 bytes per 3 pixels
Y_U_V8	planar YUV 4:4:4	8	1 byte per pixel per plane
Y_U_V10	planar YUV 4:4:4	10	4 bytes per 3 pixels per plane
Y_U_V12	planar YUV 4:4:4	12	3 bytes per 2 pixels per plane

The following tables explain the expected pixel mappings with raster scan order in memory for each of the mentioned listed formats.

RGBX8

Packed RGB, 8 bits per component. Every pixel in memory is represented with 32 bits, as shown. The images need be stored in memory in raster order, that is, top-left pixel first, bottom-right pixel last. Bits[31:24] do not contain pixel information.

31:24	23:16	15:8	7:0
X	B	G	R

YUVX8

Packed YUV 4:4:4, 8 bits per component. Every pixel in memory is represented with 32 bits, as shown. Bits[31:24] do not contain pixel information.

31:24	23:16	15:8	7:0
X	V	U	Y

BGRX8

Packed BGR, 8 bits per component. Every pixel in memory is represented with 32 bits, as shown. The images need be stored in memory in raster order, that is, top-left pixel first, bottom-right pixel last. Bits[31:24] do not contain pixel information.

31:24	23:16	15:8	7:0
X	R	G	B

YUYV8

Packed YUV 4:2:2, 8 bits per component. Every two pixels in memory are represented with 32 bits, as shown.

31:24	23:16	15:8	7:0
V0	Y1	U0	Y0

RGBA8

Packed RGB with alpha, 8 bits per component. Every pixel is represented with 32 bits, as shown. Bits[31:24] contain alpha information, 0 is fully transparent, 255 is fully opaque.

31:24	23:16	15:8	7:0
A	B	G	R

UYVY8

Packed YUV 4:2:2, 8 bits per component. Every two pixels in memory are represented with 32 bits, as shown:

31:24	23:16	15:8	7:0
Y1	V0	Y0	U0

BGRA8

Packed BGR with alpha, 8 bits per component. Every pixel is represented with 32 bits, as shown. Bits[31:24] contain alpha information, 0 is fully transparent, 255 is fully opaque.

31:24	23:16	15:8	7:0
A	R	G	B

YUVA8

Packed YUV with alpha, 8 bits per component. Every pixel is represented with 32 bits, as shown. Bits[31:24] contain alpha information, 0 is fully transparent, 255 is fully opaque.

31:24	23:16	15:8	7:0
A	V	U	Y

RGBX10

Packed RGB, 10 bits per component. Every pixel is represented with 32 bits, as shown. Bits[31:30] do not contain any pixel information.

31:30	29:20	19:10	9:0
X	B	G	R

YUVX10

Packed YUV 4:4:4, 10 bits per component. Every pixel is represented with 32 bits, as shown. Bits[31:30] do not contain any pixel information.

31:30	29:20	19:10	9:0
X	V	U	Y

RGB565

Packed RGB with 5 bits per R component, 6 bits per G component, 5 bits per B component. Every pixel is represented with 16 bits, as shown.

15:11	10:5	4:0
B	G	R

Y_U_V8

Three Planar YUV 4:4:4 with eight bits per component. Y, U, and V are stored in three separate planes, as shown in the following tables. The U plane is assumed to have an offset of stride x height bytes from the Y plane buffer address. The V plane is assumed to have an offset of stride x height bytes from the U plane buffer address.

31:24	23:16	15:8	7:0
Y3	Y2	Y1	Y0

31:24	23:16	15:8	7:0
U3	U2	U1	U0

31:24	23:16	15:8	7:0
V3	V2	V1	V0

Y_UV8

Semi-planar YUV 4:2:2 with 8 bits per component. Y and UV stored in separate planes as shown. The UV plane is assumed to have an offset of stride × height bytes from the Y plane buffer address.

31:24	23:16	15:8	7:0
Y3	Y2	Y1	Y0

31:24	23:16	15:8	7:0
V2	U2	V0	U0

Y_UV8_420

Semi-planar YUV 4:2:0 with 8 bits per component. Y and UV stored in separate planes as shown. The UV plane is assumed to have an offset of stride × height bytes from the Y plane buffer address.

31:24	23:16	15:8	7:0
Y3	Y2	Y1	Y0

31:24	23:16	15:8	7:0
V4	U4	V0	U0

RGB8

Packed RGB, 8 bits per component. Every pixel in memory is represented with 24 bits, as shown. The images need be stored in memory in raster order, that is, top-left pixel first, bottom-right pixel last.

23:16	15:8	7:0
B	G	R

BGR8

Packed BGRB, 8 bits per component. Every pixel in memory is represented with 24 bits, as shown. The images need be stored in memory in raster order, that is, top-left pixel first, bottom-right pixel last.

23:16	15:8	7:0
R	G	B

YUV8

Packed YUV 4:4:4, 8 bits per component. Every pixel in memory is represented with 24 bits, as shown. The images need be stored in memory in raster order, that is, top-left pixel first, bottom-right pixel last.

23:16	15:8	7:0
V	U	Y

Y_UV10

Semi-planar YUV 4:2:2 with 10 bits per component. Every 3 pixels is represented with 32 bits. Bits[31:30] do not contain any pixel information. Y and UV stored in separate planes as shown. The UV plane is assumed to have an offset of stride x height bytes from the Y plane buffer address.

63:62	61:52	51:42	41:32	31:30	29:20	19:10	9:0
X	Y5	Y4	Y3	X	Y2	Y1	Y0

63:62	61:52	51:42	41:32	31:30	29:20	19:10	9:0
X	V4	U4	V2	X	U2	V0	U0

Y_UV10_420

Semi-planar YUV 4:2:0 with 10 bits per component. Every 3 pixels is represented with 32 bits. Bits[31:30] do not contain any pixel information. Y and UV stored in separate planes as shown. The UV plane is assumed to have an offset of stride x height bytes from the Y plane buffer address.

63:62	61:52	51:42	41:32	31:30	29:20	19:10	9:0
X	Y5	Y4	Y3	X	Y2	Y1	Y0

63:62	61:52	51:42	41:32	31:30	29:20	19:10	9:0
X	V8	U8	V4	X	U4	V0	U0

Y_U_V10

Three Planar YUV 4:4:4 with 10 bits per component. Every three pixels is represented with 32 bits. Bits[31:30] do not contain any pixel information. Y, U, and V are stored in three separate planes as shown. The U plane is assumed to have an offset of stride x height bytes from the Y plane buffer address. The V plane is assumed to have an offset of stride x height bytes from the U plane buffer address.

63:62	61:52	51:42	41:32	31:30	29:20	19:10	9:0

X	Y5	Y4	Y3	X	Y2	Y1	Y0

63:62	61:52	51:42	41:32	31:30	29:20	19:10	9:0
X	U5	U4	U3	X	U2	U1	U0

63:62	61:52	51:42	41:32	31:30	29:20	19:10	9:0
X	V5	V4	V3	X	V2	V1	V0

Y8

Packed Luma-Only, 8 bits per component. Every luma-only pixel in memory is represented with 8 bits, as shown. The images need be stored in memory in raster order, that is, top-left pixel first, bottom-right pixel last. Y8 is presented as YUV 4:4:4 on the AXI4-Stream interface.

7:0
Y

Y10

Packed Luma-Only, 10 bits per component. Every three luma-only pixels in memory is represented with 32 bits, as shown. The images need be stored in memory in raster order, that is, top-left pixel first, bottom-right pixel last. Y10 is presented as YUV 4:4:4 on the AXI4-Stream interface.

31:30	29:20	19:10	9:0
X	Y2	Y1	Y0

Y_U_V12

Three Planar YUV 4:4:4 with 12 bits per component. Y, U, and V are stored in three separate planes, as shown in the following tables. The U plane is assumed to have an offset of stride x height bytes from the Y plane buffer address. The V plane is assumed to have an offset of stride x height bytes from the U plane buffer address.

47:36	35:24	23:12	11:0
Y3	Y2	Y1	Y0

47:36	35:24	23:12	11:0
U3	U2	U1	U0

47:36	35:24	23:12	11:0
V3	V2	V1	V0

Tile Format

Tiling is a method of re-arranging the pixels of a surface so that pixels that are close in the two-dimensional euclidean space are more likely to be stored close to each other in the memory. A tiled image is formed by dividing the image pixels into two-dimensional blocks or tiles. Each tile takes up a chunk of contiguous memory. Tiles are stored in raster scan order from left to right and from top to bottom within each plane.

$Tile_0$	$Tile_1$	$Tile_2$	...	$Tile_{N-2}$	$Tile_{N-1}$	
$Tile_N$	$Tile_{N+1}$	$Tile_{N+2}$	...	$Tile_{2N-2}$	$Tile_{2N-1}$	
$Tile_{2N}$	$Tile_{2N+1}$	$Tile_{2N+2}$	...	$Tile_{3N-2}$	$Tile_{3N-1}$	
...	...	...	...	...	...	

***Note*:** Tile Format is supported only in the AMD Versal™ AI Edge Gen 2 and AMD Versal™ Prime Gen 2 series devices where Video Codec Unit (VCU2) AMD LogiCORE™ is available as a Hard IP. For more information, see *Versal AI Edge Gen 2 H.264/H.265/JPEG Video Codec Unit 2 Solutions LogiCORE IP Product Guide* (PG447).

The frame buffer read and write IPs support memory video format in two tile sizes: 32 × 4 and 64 × 4.

- **32 × 4 Tiled Format:**

  With B corresponding to a block of 4 × 4 pixel components, the sample order within a 32 × 4 tile corresponds to a sequence of eight 4 × 4 blocks:

B0	B1	B2	B3	B4	B5	B6	B7

- **64 × 4 Tiled Format:**

  The sample order within a 64 × 4 tile corresponds to a sequence of sixteen 4 × 4 blocks:

B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15

  The sample order within each 4 × 4 block B corresponds to the raster scan order.

  - **Luma:**

    The order of pixel components within a 4 × 4 block in luma plane is shown below:

$Y_{x,0}$	$Y_{x,1}$	$Y_{x,2}$	$Y_{x,3}$
$Y_{x,4}$	$Y_{x,5}$	$Y_{x,6}$	$Y_{x,7}$
$Y_{x,8}$	$Y_{x,9}$	$Y_{x,A}$	$Y_{x,B}$
$Y_{x,C}$	$Y_{x,D}$	$Y_{x,E}$	$Y_{x,F}$

  - **Chroma Non-Interleaved:**

With C corresponding to either Cb or Cr block, the order of pixel components within a 4 × 4 block in Cb or Cr plane is shown below:

$C_{x,0}$	$C_{x,1}$	$C_{x,2}$	$C_{x,3}$
$C_{x,4}$	$C_{x,5}$	$C_{x,6}$	$C_{x,7}$
$C_{x,8}$	$C_{x,9}$	$C_{x,A}$	$C_{x,B}$
$C_{x,C}$	$C_{x,D}$	$C_{x,E}$	$C_{x,F}$

- **Chroma Interleaved:**

  The order of pixel components within a 4 × 4 block in interleaved Chroma plane is shown below:

$Cb_{x,0}$	$Cr_{x,0}$	$Cb_{x,1}$	$Cr_{x,1}$
$Cb_{x,2}$	$Cr_{x,2}$	$Cb_{x,3}$	$Cr_{x,3}$
$Cb_{x,4}$	$Cr_{x,4}$	$Cb_{x,5}$	$Cr_{x,5}$
$Cb_{x,6}$	$Cr_{x,6}$	$Cb_{x,7}$	$Cr_{x,7}$

  Consequently, in a 4:2:0 picture, there are half as many chroma tiles as luma tiles because the chroma plane has half the vertical size of the luma plane besides having the same horizontal size, as Cb and Cr components are interleaved. In 4:4:4 and 4:2:2 pictures, there are equal number of chroma and luma tiles. The 4:0:0 monochrome picture consists of only luma tiles.

  The pixel components within each tile are fully packed so the memory footprint of a tile is:

  - 128 bytes for an 8-bit 32 × 4 tile
  - 160 bytes for a 10-bit 32 × 4 tile
  - 192 bytes for a 12-bit 32 × 4 tile
  - 256 bytes for an 8-bit 64 × 4 tile
  - 320 bytes for a 10-bit 64 × 4 tile
  - 384 bytes for a 12-bit 64 × 4 tile

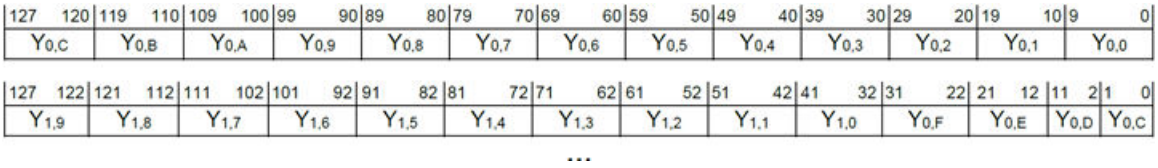
The following tables explain the expected pixel mappings in memory for each of the supported tile formats.

- **8-bit Tile Format:** 8-bit samples are packed starting from the least significant bits of memory words. The footprint of an 8-bit 32x4 tile is 128 bytes (8 x 128-bit words) and the footprint of an 8-bit 64x4 tile is 256 bytes (16 x 128-bit words). An example of tile buffer for luma samples is as follows:

127 120	119 112	111 104	103 96	95 88	87 80	79 72	71 64	63 56	55 48	47 40	39 32	31 24	23 16	15 8	7 0
$Y_{0,F}$	$Y_{0,E}$	$Y_{0,D}$	$Y_{0,C}$	$Y_{0,B}$	$Y_{0,A}$	$Y_{0,9}$	$Y_{0,8}$	$Y_{0,7}$	$Y_{0,6}$	$Y_{0,5}$	$Y_{0,4}$	$Y_{0,3}$	$Y_{0,2}$	$Y_{0,1}$	$Y_{0,0}$
$Y_{1,F}$	$Y_{1,E}$	$Y_{1,D}$	$Y_{1,C}$	$Y_{1,B}$	$Y_{1,A}$	$Y_{1,9}$	$Y_{1,8}$	$Y_{1,7}$	$Y_{1,6}$	$Y_{1,5}$	$Y_{1,4}$	$Y_{1,3}$	$Y_{1,2}$	$Y_{1,1}$	$Y_{1,0}$

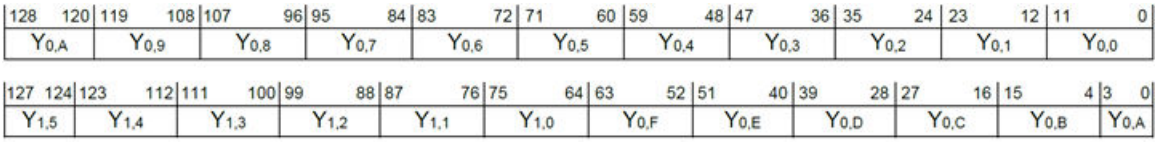
...

- **10-bit Tile Format:** 10-bit samples are packed starting from the least significant bits of memory words. The footprint of a 10-bit 32x4 tile is 160 bytes (10 x 128-bit words) and the footprint of a 10-bit 64x4 tile is 320 bytes (20 x 128-bit words). An example of tile buffer for luma samples is as follows:



...

- **12-bit Tile Format:** 12-bit samples are packed starting from the least significant bits of memory words. The footprint of a 12-bit 32x4 tile is 192 bytes (12 x 128-bit words) and the footprint of a 12-bit 64x4 tile is 384 bytes (24 x 128-bit words). An example of tile buffer for luma samples is as follows:



...

The following table shows the pixel formats with tile scan order in memory supported by the Video Mixer.

Table 9: **Pixel Formats with Tile Scan Order**

Video Format	Description	Bits per Component	Bytes per 32x4 Tile	Bytes per 64x4 Tile
Y_U_V8	3 planar YUV 4:4:4	8	128	256
Y_U_V10	3 planar YUV 4:4:4	10	160	320
Y_U_V12	3 planar YUV 4:4:4	12	192	384
Y_UV8	semi-planar YUV 4:2:2	8	128	256
Y_UV10	semi-planar YUV 4:2:2	10	160	320
Y_UV12	semi-planar YUV 4:2:2	12	192	384
Y_UV8_420	semi-planar YUV 4:2:0	8	128	256
Y_UV10_420	semi-planar YUV 4:2:0	10	160	320
Y_UV12_420	semi-planar YUV 4:2:0	12	192	384
Y8	monochrome YUV 4:0:0	8	128	256
Y10	monochrome YUV 4:0:0	10	160	320
Y12	monochrome YUV 4:0:0	12	192	384

AXI4-Lite Control Interface

The AXI4-Lite interface allows you to dynamically control parameters within the core. The configuration can be accomplished using an AXI4-Lite master state machine, an embedded Arm®, or soft system processor such as AMD MicroBlaze™. The Video Mixer can be controlled through the AXI4-Lite interface by using functions provided by the driver in the AMD Vitis™ tools. Another method is performing read and write transactions to the register space but should only be used when the first method is not available. The following table shows the AXI4-Lite control interface signals. This interface runs at the `ap_clk` clock.

Table 10: **AXI4-Lite Control Interface Signals**

Signal Name	I/O	Width	Description
s_axi_ctrl_aresetn	I	1	Reset
s_axi_ctrl_aclk	I	1	Clock
s_axi_ctrl_awaddr	I	18	Write Address
s_axi_ctrl_awprot	I	3	Write Address Protection
s_axi_ctrl_awvalid	I	1	Write Address Valid
s_axi_ctrl_awready	O	1	Write Address Ready
s_axi_ctrl_wdata	I	32	Write Data
s_axi_ctrl_wstrb	I	4	Write Data Strobe
s_axi_ctrl_wvalid	I	1	Write Data Valid
s_axi_ctrl_wready	O	1	Write Data Ready
s_axi_ctrl_bresp	O	2	Write Response
s_axi_ctrl_bvalid	O	1	Write Response Valid
s_axi_ctrl_bready	I	1	Write Response Ready
s_axi_ctrl_araddr	I	18	Read Address
s_axi_ctrl_arprot	I	3	Read Address Protection
s_axi_ctrl_arvalid	I	1	Read Address Valid
s_axi_ctrl_aready	O	1	Read Address Ready
s_axi_ctrl_rdata	O	32	Read Data
s_axi_ctrl_rresp	O	2	Read Data Response
s_axi_ctrl_rvalid	O	1	Read Data Valid
s_axi_ctrl_rready	I	1	Read Data Ready

Register Space

The Video Mixer has specific registers which allow you to dynamically control the operation of the core. All registers have an initial value of 0.

Top-Level Registers

The following table provides a detailed description of all the registers that apply globally to the IP.

Table 11: **Top-Level Registers**

Address (hex) BASEADDR+	Register Name	Access Type	Register Description
0x0000	Control	R/W	Bit[0] = ap_start (R/W/COH) [1] Bit[1] = ap_done (R/COR)[1] Bit[2] = ap_idle (R) Bit[3] = ap_ready (R) Bit[5] = Flush pending AXI transactions (R/W)[2] Bit[6] = Flush done (R) Bit[7] = auto_restart (R/W) Others = reserved
0x0004	Global Interrupt Enable	R/W	Bit[0] = Global interrupt enable Others = Reserved
0x0008	IP Interrupt Enable		Bit[0] = ap_done Bit[1] = ap_ready Others = reserved
0x000C	IP Interrupt Status Register	R/TOW[1]	Bit[0] = ap_done Bit[1] = ap_ready Others = Reserved
0x0010	Width	R/W	Active width of background.
0x0018	Height	R/W	Active height of background.
0x0028	Background R or Y	R/W	Red or Y value of background color
0x0030	Background U or G	R/W	Green or U value of background color
0x0038	Background V or B	R/W	Blue or V value of background color
0x0040	Layer enable	R/W	Bit[0] = Master layer is enabled/disabled Bit[1] = Overlay Layer 1 is enabled/disabled ... Bit[16] = Overlay Layer 16 is enabled/disabled Bit[23] = Logo layer is enabled/disabled

Notes:

1. TOW = Toggle on Write, COH = Clear on Handshake, COR = Clear on Read.
2. BIT[5] and BIT[6] are applicable only for memory based layers.
3. Control Register (0x0000), Global Interrupt Enable Register (0x0004), IP Interrupt Enable Register (0x0008), and IP Interrupt Status Register (0x000C) are explained in section S_AXILITE Control Register Map of *Vitis High-Level Synthesis User Guide* (UG1399). These registers' definitions might have some additional bits; however, in the current IP, you can access only the bits mentioned in the preceding table. Therefore, only these bits need to be considered while accessing the above registers.

Control (0x0000) Register

This register controls the operation of the core.

- Bit[0] of the Control register, `ap_start`, kicks off the core from software. Writing 1 to this bit, starts the core to generate a video frame.
- Bit[1] of the Control register, `ap_done`, indicates when the IP has completed all operations in the current transaction. A logic 1 on this signal indicates that the IP has completed all operations in this transaction.
- Bit[2] of the Control register, `ap_idle`, signal indicates if the IP is operating or idle (no operation). The idle state is indicated by logic 1. This signal is asserted low after the IP starts operating. This signal is asserted high when the IP completes operation and no further operations are performed.
- Bit[3] of the Control register, `ap_ready`, signal indicates when the IP is ready for new inputs. It is set to logic 1 when the IP is ready to accept new inputs, indicating that all input reads for this transaction are completed. If the IP has no operations in the pipeline, new reads are not performed until the next transaction starts. This signal is used to make a decision on when to apply new values to the input ports and whether to start a new transaction.
- Bit[5] of the Control register, is for flushing pending AXI transactions. This bit should be set and held (until reset) by software to flush pending transactions. When this is set, the hardware is expecting a hard reset.
- Bit[6] of the Control register is the flush status bit and is asserted when the flush is done.
- Bit[7] of the Control register, `auto_restart`, can be set to enable the auto-restart mode. Thereafter, the IP restarts automatically at the end of each transaction.

Global Interrupt Enable (0x0004) Register

This register is the master control for all interrupts. Bit[0] can be used to enable/disable all core interrupts.

IP Interrupt Enable (0x0008) Register

This register allows interrupts to be enabled selectively. Currently, two interrupt sources are available `ap_done` and `ap_ready`. `ap_done` is triggered after the frame processing is complete, while `ap_ready` is triggered after the core is ready to start processing the next frame.

IP Interrupt Status (0x000C) Register

This is a dual purpose register. When an interrupt occurs, the corresponding interrupt source bit is set in this register. In readback mode (Get status), the interrupting source can be determined. In writeback mode (Clear interrupt), the requested interrupt source bit is cleared.

Width (0x0010) Register

The WIDTH register encodes the number of active pixels per scan. Supported values are between 64 and the value provided in the Maximum Number of Columns field in the Vivado Integrated Design Environment (IDE). To avoid processing errors, you should restrict values written to width to the range supported by the core instance. Furthermore, width needs to be a multiple of the Samples per Clock field in the AMD Vivado™ IDE.

Height (0x0018) Register

The Height register encodes the number of active scan lines per frame. Supported values are between 64 and the value provided in the Maximum Number of Rows field in the Vivado IDE. To avoid processing errors, you should restrict values written to height to the range supported by the core instance.

Background Y or R (0x0028) Register

Red color component of the background color. The range of the background color is determined by the number of bits per color component selected in the Maximum Data Width field in the Vivado IDE, for example, 0..1023 for 10-bit maximum data width.

Background U or G (0x0030) Register

Green color component of the background color. The range of the background color is determined by the number of bits per color component selected in the Maximum Data Width field in the Vivado IDE, for example, 0..1023 for 10-bit maximum data width.

Background V or B (0x0038) Register

Blue color component of the background color. The range of the background color is determined by the number of bits per color component selected in the Maximum Data Width field in the Vivado IDE, for example, 0..1023 for 10-bit maximum data width.

Layer Enable (0x0040) Register

This register has one bit for every layer that indicates whether a layer is enabled (1) or disabled (0). Bit[0] is for the master input layer, which associates with the `s_axis_video` AXI4-Stream input. This is the bottom-most layer, and all other layers are blended on top. If this layer is disabled and no other layers are blended on top, the background color as specified by the previous mentioned registers are shown. If this layer is enabled, video data should be sent to this layer, otherwise the Video Mixer stalls.

Bit[1] is for overlay layer 1, which associates with either `s_axis_video1` AXI4-Stream input or `m_axi_mm_video1` memory mapped AXI4 input, depending on the Interface Type field selected in the Vivado IDE. Bit[2] is for overlay layer 2 and so on, until Bit[16] which enables/disables layer 16.

Bit[23] is for the logo layer. The logo layer is the top-most layer and is blended on top of all other layers.

CSC Coefficient Registers

The following table provides a detailed description of CSC coefficient registers:

***Note*:** These registers are available to program only if the Enable CSC Coefficient Registers option is enabled in the GUI. With these registers, different CSC coefficients can be programmed to support BT601, BT709, and BT2020.

Table 12: **CSC Coefficient Registers**

Address (hex) BASEADDR+	Register Name	Access Type	Description
0x00048	Coefficient K 11 for RGB	R/W	16 Bits K11[15:0] are Read/Write and others are reserved
0x00050	Coefficient K 12 for RGB	R/W	16 Bits K12[15:0] are Read/Write and others are reserved
0x00058	Coefficient K 13 for RGB	R/W	16 Bits K13[15:0] are Read/Write and others are reserved
0x00060	Coefficient K 21 for RGB	R/W	16 Bits K21[15:0] are Read/Write and others are reserved
0x00068	Coefficient K 22 for RGB	R/W	16 Bits K22[15:0] are Read/Write and others are reserved
0x00070	Coefficient K 23 for RGB	R/W	16 Bits K23[15:0] are Read/Write and others are reserved
0x00078	Coefficient K 31 for RGB	R/W	16 Bits K31[15:0] are Read/Write and others are reserved
0x00080	Coefficient K 32 for RGB	R/W	16 Bits K32[15:0] are Read/Write and others are reserved
0x00088	Coefficient K 33 for RGB	R/W	16 Bits K33[15:0] are Read/Write and others are reserved
0x00090	R Offset	R/W	12 Bits ROffset[11:0] are Read/Write and others are reserved
0x00098	G Offset	R/W	12 Bits GOffset[11:0] are Read/Write and others are reserved
0x000A0	B Offset	R/W	12 Bits BOffset[11:0] are Read/Write and others are reserved
0x00140	Coefficient K11 for YUV	R/W	16 Bits K11_2[15:0] are Read/Write and others are reserved
0x00148	Coefficient K12 for YUV	R/W	16 Bits K12_2[15:0] are Read/Write and others are reserved
0x00150	Coefficient K13 for YUV	R/W	16 Bits K13_2[15:0] are Read/Write and others are reserved

Table 12: **CSC Coefficient Registers** *(cont'd)*

Address (hex) BASEADDR+	Register Name	Access Type	Description
0x00158	Coefficient K21 for YUV	R/W	16 Bits K21_2[15:0] are Read/Write and others are reserved
0x00160	Coefficient K22 for YUV	R/W	16 Bits K22_2[15:0] are Read/Write and others are reserved
0x00168	Coefficient K23 for YUV	R/W	16 Bits K23_2[15:0] are Read/Write and others are reserved
0x00170	Coefficient K31 for YUV	R/W	16 Bits K31_2[15:0] are Read/Write and others are reserved
0x00178	Coefficient K32 for YUV	R/W	16 Bits K32_2[15:0] are Read/Write and others are reserved
0x00180	Coefficient K33 for YUV	R/W	16 Bits K33_2[15:0] are Read/Write and others are reserved
0x00188	Y Offset	R/W	12 Bits YOffset[11:0] are Read/Write and others are reserved
0x00190	U Offset	R/W	12 Bits UOffset[11:0] are Read/Write and others are reserved
0x00198	V Offset	R/W	12 Bits VOffset[11:0] are Read/Write and others are reserved

Notes:

1. These registers can be used to program your coefficients.
2. For YUV to RGB and RGB to YUV equations, refer to CSC section of the *Video Processing Subsystem Product Guide* (PG231).

Layer Registers

The following table provides a detailed description of all the registers that apply to layers 0 through 16. Overlay layer 1 registers start at base address 0x0200, overlay layer 2 registers start at base address 0x300, and so forth.

Mixer IP has one primary layer, which we mention as layer0. It supports up to 16 overlay layers (Layer 1 to Layer 16). Layer 0 does not have any register for programming. For all overlay layers (Layer 1 to Layer 16) the registers are at identical offsets. Layer 1 registers start at base address 0x200, Layer 2 registers start at base address 0x300 and so forth. Layer 16 registers start at base address 0x1100. In this section, the register description number represents a number from 2 to 11.

Table 13: **Layer Registers**

Address (hex) BASEADDR+	Register Name	Access Type	Description
0x0200	Layer 1 Alpha	R/W	Alpha blending value for layer 1 ranging from 0 = Fully transparent 256 = Fully opaque

Table 13: **Layer Registers** *(cont'd)*

Address (hex) BASEADDR+	Register Name	Access Type	Description
0x0208	Layer 1 Start X	R/W	X position of the top left corner of layer 1, relative to the background layer.
0x0210	Layer 1 Start Y	R/W	Y position of the top left corner of layer 1, relative to the background layer
0x0218	Layer 1 Width	R/W	Active width (in pixels) of layer 1.
0x0220	Layer 1 Stride	R/W	Active stride (in bytes) of layer 1.
0x0228	Layer 1 Height	R/W	Active height (in lines) of layer 1.
0x0230	Layer 1 Scale Factor	R/W	Scale factor for layer 1 ranging from 0 = No scaling 1 = 2x scaling (horizontally and vertically) 2 = 4x scaling (horizontally and vertically)
0x0240	Layer 1 Plane 1 Buffer	R/W	Start address of plane 1 of frame buffer for layer 1. Only valid in case layer 1 is a memory layer.
0x024C	Layer 1 Plane 2 Buffer	R/W	Start address of plane 2 of frame buffer for layer 1. Only valid in case layer 1 is a memory layer and a semi-planar video format is selected.

Layer Alpha (0x0#00) Register

The Layer Alpha register specifies the per layer alpha blending value used for blending this layer with the underlying layer. The value of this register has a range from 0 which is fully transparent, to 256 which is fully opaque. Layer alpha blending is only supported when the Enable Global Alpha field in the Vivado IDE is enabled for a particular layer. When alpha blending is disabled for a layer, layers are blended as fully opaque on top of the underlying layer.

Layer Start X (0x0#08) Register

The Layer Start X register marks the x position (columns) of the top left corner of the layer relative to the mixer output frame dimensions. (0,0) puts the layer in the top left corner of the output frame. To avoid processing errors, you should restrict values written to start x such that the entire layer is contained within the frame. Furthermore, the x position needs to be a multiple of the Samples per Clock field in the Vivado IDE.

Layer Start Y (0x0#10) Register

The Layer Start Y register marks the y position (rows) of the top left corner of the layer relative to the mixer output frame dimensions. (0,0) puts the layer in the top left corner of the output frame. To avoid processing errors, you should restrict values written to start y such that the entire layer is contained within the frame.

Layer Width (0x0#18) Register

The Layer Width register encodes the active width in pixels of the layer. If Maximum Number of Columns field in Vivado IDE is less than or equal to 4096, the supported values are between 64 and Maximum Number of Columns. If the Maximum Number of Columns is more than 4096, the supported values are between 64 and 4096. It is between 64 and the Layer Line Buffer Width in the Vivado IDE if the Enable Scaling field is enabled. To avoid processing errors, you should restrict values written to layer width such that the entire layer is contained within the frame. Furthermore, layer width needs to be a multiple of the Samples per Clock field in the Vivado IDE for the raster scan order, and a multiple of both four and the Samples per Clock field for the tiled order.

Layer Stride (0x0#20) Register

This register is not applicable if the layer is a streaming layer. If it is a memory layer, the layer stride determines the number of bytes between rows of pixels or rows of tiles in memory based on the following memory video formats:

- **Memory in Raster Order:** The stride value in raster order defines the address offset between two consecutive rows of pixels.
- **Memory in Tile Order:** The stride value in tile order defines the address offset between two consecutive rows of tiles. A row of tiles corresponds to four rows of pixels. The stride in tile order is also referred to as Pitch.

When a video frame is stored in memory, the memory buffer might contain extra padding bytes after each row of pixels. The padding bytes only affect how the image is stored in memory, but does not affect how the image is displayed.

Padding bytes are necessary to make sure that every row of pixels starts at an address that is aligned with the size of the data on the memory mapped AXI4 interface. Therefore, layer stride needs to be a multiple of the memory mapped AXI4 data size. For the Video Mixer, the data size of the memory mapped AXI4 interface is 64 × Samples per Clock bits, that is, 64, 128, 256, and 512 bits for 1, 2, 4, and 8 samples per clock, respectively. In tile format, padding bytes are also necessary when the width of the video frame is not exactly divisible by the width of the tile.

Layer Height (0x0#28) Register

The Layer Height register encodes the height in lines of the layer. Supported values are between 64 and the value provided in the Maximum Number of Rows field in the Vivado IDE. To avoid processing errors, you should restrict values written to layer height such that the entire layer, after applying the scale factor, is contained within the frame. Furthermore, layer height needs to be a multiple of eight in case of tile scan order.

Layer Scale Factor (0x0#30) Register

The Layer Scale Factor register determines the scaling factor that is applied before blending this layer with the underlying layer. Scale factors of 1x, 2x, and 4x are supported.

- 0 = No scaling
- 1 = 2x scaling (horizontally and vertically)
- 2 = 4x scaling (horizontally and vertically)

Layer scaling is only supported when the Enable Scaling field in the Vivado IDE is enabled for a particular layer.

Layer Buffer Plane 1 (0x0#40), Layer Plane 2 Buffer (0x0#4C), and Layer Plane 3 Buffer (0x0#58) Registers

In case the layer is a memory layer, the Layer Plane 1 Buffer register specifies the frame buffer address of plane 1. For the semi-planar formats (Y_UV8, Y_UV8_420, Y_UV10, and Y_UV10_420), the chroma buffer is specified by the Layer Plane 2 Buffer register. For the planar formats (Y_U_V8, Y_U_V10, and Y_U_V12), the U chroma buffer is specified by the Layer Plane 2 Buffer register and V chroma buffer is specified by the Layer Plane 3 Buffer register. The addresses must be aligned to the data size of the memory mapped AXI4 interface. For the Video Mixer, the data size of the memory mapped AXI4 interface is 64 × Samples per Clock bits, that is, 64, 128, 256, and 512 bits for 1, 2, 4, and 8 samples per clock, respectively. These registers are not applicable when the layer is a streaming layer.

Logo Layer Registers

The following table provides a detailed description of all the registers that apply to the logo layer.

Table 14: **Logo Layer Registers**

Address (hex) BASEADDR+	Register Name	Access Type	Description
0x2000	Logo Start X	R/W	X position of the top left corner of the logo, relative to the output resolution
0x2008	Logo Start Y	R/W	Y position of the top left corner of the logo, relative to the output resolution
0x2010	Logo Width	R/W	Active width (in pixels) of the logo
0x2018	Logo Height	R/W	Active height (in lines) of the logo
0x2020	Logo Scale Factor	R/W	Scale factor for logo ranging from 0 = No scaling 1 = 2x scaling (horizontally and vertically) 2 = 4x scaling (horizontally and vertically)

Table 14: **Logo Layer Registers** *(cont'd)*

Address (hex) BASEADDR+	Register Name	Access Type	Description
0x2028	Logo Alpha	R/W	Alpha blending value for logo ranging from 0 = Fully transparent 255 = Fully opaque
0x2030	Logo Color Key Min R	R/W	Red minimum value of color key range
0x2038	Logo Color Key Min G	R/W	Green minimum value of color key range
0x2040	Logo Color Key Min B	R/W	Blue minimum value of color key range
0x2048	Logo Color Key Max R	R/W	Red maximum value of color key range
0x2050	Logo Color Key Max G	R/W	Green maximum value of color key range
0x2058	Logo Color Key Max B	R/W	Blue maximum value of color key range
0x1 0000	Logo Red Buffer	R/W	Start address of buffer for red logo pixels
0x2 0000	Logo Green Buffer	R/W	Start address of buffer for green logo pixels
0x3 0000	Logo Blue Buffer	R/W	Start address of buffer for blue logo pixels
0x4 0000	Logo Alpha Buffer	R/W	Start address of buffer for logo per pixel alpha

Logo Start X (0x2000) Register

The Logo Start X register marks the x position (columns) of top left corner of the logo relative to the mixer output frame dimensions. (0,0) puts the logo in the top left corner of the output frame. To avoid processing errors, you should restrict values written to start x such that the entire logo is contained within the frame. Furthermore, the x position needs to be a multiple of the Samples per Clock field in the Vivado IDE.

Logo Start Y (0x2008) Register

The Logo Start Y register marks the y position (rows) of the top left corner of the logo relative to the mixer output frame dimensions. (0,0) puts the logo in the top left corner of the output frame. To avoid processing errors, you should restrict values written to start y such that the entire logo is contained within the frame.

Logo Width (0x2010) Register

The Logo Width register encodes the width in pixels of the logo. Supported values are between 32 and the value provided in the Maximum Number of Columns for Logo field in the Vivado IDE. To avoid processing errors, you should restrict values written to width to the range supported by the core instance. Furthermore, width needs to be a multiple of the Samples per Clock field in the Vivado IDE.

Logo Height (0x2018) Register

The Logo Height register encodes the height in lines of the logo. Supported values are between 32 and the value provided in the Maximum Number of Rows for Logo field in the Vivado IDE. To avoid processing errors, you should restrict values written to height to the range supported by the core instance.

Logo Scale Factor (0x2020) Register

The Logo Scale Factor register determines the scaling factor that is applied before blending the logo with the underlying layer. Scale factors of 1x, 2x, and 4x are supported.

- 0 = No scaling
- 1 = 2x scaling (horizontally and vertically)
- 2 = 4x scaling (horizontally and vertically)

Logo Alpha (0x2028) Register

The Logo Alpha register specifies the per layer alpha blending value used for blending the logo with the underlying layer. The value of this register has a range from 0 which is fully transparent, to 256 which is fully opaque.

Logo Color Key Min R (0x2030) Register

The Logo Color Key values determine a color range that is treated as transparent. Any logo pixel that falls in this range are not blended on top of the underlying layer but is transparent. This equation is used to determine if a pixel is transparent.

Let (`r, g, b`) be a pixel value of the logo.

Let (`min_r, min_g, min_b`) and (`max_r, max_g, max_b`) be the values of the Color Key registers.

Then, pixel (`r, g, b`) is transparent if the following holds:

`min_r ≤ r ≤ max_r` AND

`min_g ≤ g ≤ max_g` AND

`min_b ≤ b ≤ max_b`

To turn off background color keying, program a maximum value that is smaller than the minimum value, for example, maximum is 0, minimum is 1.

Logo Color Key Min G (0x2038)

For more information, see Logo Color Key Min R (0x2030) Register.

Register, Logo Color Key Min B (0x2040) Register

For more information, see Logo Color Key Min R (0x2030) Register.

Logo Color Key Max R (0x2048) Register

For more information, see Logo Color Key Min R (0x2030) Register.

Logo Color Key Max G (0x2050) Register

For more information, see Logo Color Key Min R (0x2030) Register.

Logo Color Key Max B (0x2058) Register

For more information, see Logo Color Key Min R (0x2030) Register.

Logo Red Buffer (0x1 0000) Register

The memory for storing the frame data for the logo layer is block RAM local to the core. The application is required to load the logo into this block RAM through the AXI4-Lite interface. The Logo Buffer address is the start address for this block RAM. The Logo Buffer addresses point to block RAM for separate red, green, and blue pixel buffers that hold the logo pixels. Optionally, if Logo Per Pixel Alpha is enabled, there is an additional buffer for the logo per pixel alpha values. The logo has to be formatted as planar RGB(A) with eight bits per color component. The size of the buffer is dimensioned by the Maximum Number of Columns for Logo and the Maximum Number of Rows for Logo fields in the Vivado IDE. With eight bits per color component, the size in bytes is defined as columns × rows. The logo pixels have to be organized in the buffer as follows.

- Red[x + y × columns] holds the red pixel of the logo for column x and row y
- Green[x + y × columns] holds the green pixel of the logo for column x and row y
- Blue[x + y × columns] holds the blue pixel of the logo for column x and row y
- Alpha[x + y × columns] holds the per pixel alpha value of the logo for column x and row y

Logo Green Buffer (0x2 0000) Register

For more information, see Logo Red Buffer (0x1 0000) Register.

Logo Blue Buffer (0x3 0000) Register

For more information, see Logo Red Buffer (0x1 0000) Register.

Logo Alpha Buffer (0x4 0000) Register

For more information, see Logo Red Buffer (0x1 0000) Register.

***Note*:** The logo layer requires 512 Kb address space in Vivado address map to access logo layer registers.

Design Flow Steps

This section describes customizing and generating the core, constraining the core, and the simulation, synthesis, and implementation steps that are specific to this IP core. More detailed information about the standard AMD Vivado™ design flows and the IP integrator can be found in the following Vivado Design Suite user guides:

- *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* (UG994)
- *Vivado Design Suite User Guide: Designing with IP* (UG896)
- *Vivado Design Suite User Guide: Getting Started* (UG910)
- *Vivado Design Suite User Guide: Logic Simulation* (UG900)

Customizing and Generating the Core

This section includes information about using AMD tools to customize and generate the core in the Vivado Design Suite.

If you are customizing and generating the core in the Vivado IP integrator, see the *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* (UG994) for detailed information. IP integrator might auto-compute certain configuration values when validating or generating the design. To check whether the values do change, see the description of the parameter in this chapter. To view the parameter value, run the `validate_bd_design` command in the Tcl console.

You can customize the IP for use in your design by specifying values for the various parameters associated with the IP core using the following steps:

1. Select the IP from the Vivado IP catalog.
2. Double-click the selected IP or select the **Customize IP** command from the toolbar or right-click menu.

For details, see the *Vivado Design Suite User Guide: Designing with IP* (UG896) and the *Vivado Design Suite User Guide: Getting Started* (UG910).

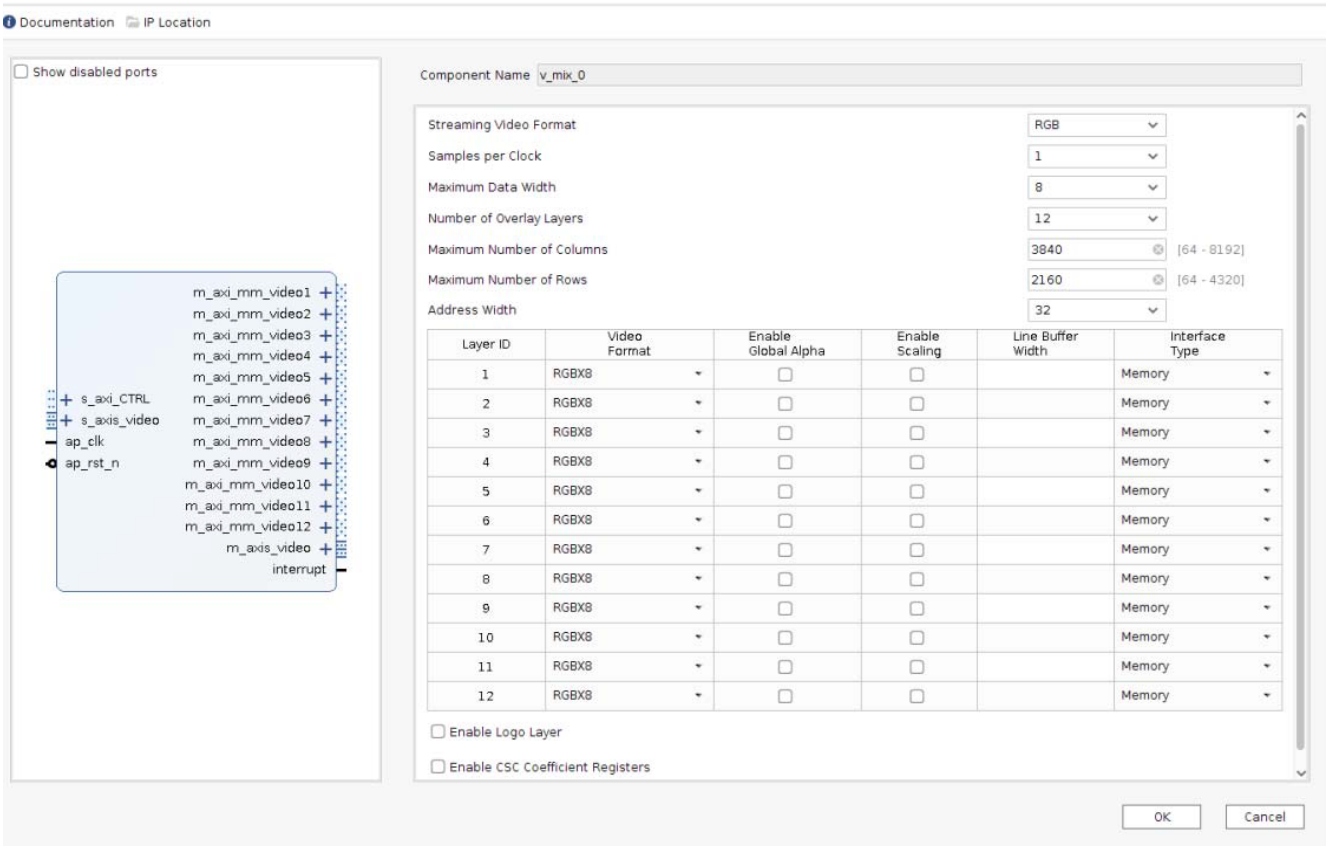
Note: Figure in this chapter is an illustration of the Vivado Integrated Design Environment (IDE). The layout depicted here might vary from the current version.

Interface

The Video Mixer is configured to meet your specific needs through the Vivado Design Suite. This section provides a quick reference to parameters that can be configured at generation time.

The following figure shows the Video Mixer Vivado IDE main configuration screen.

Figure 3: **Video Mixer Customize IP**



General Settings

The following settings are generally applicable:

- **Component Name:** The component name is used as the base name of output files generated for the module. Names must begin with a letter and must be composed from characters: a to z, 0 to 9 and "_".
- **Streaming Video Format:** Specifies the video format on the streaming input (`s_axis_video`) and output (`m_axis_video`) AXI4-Stream interfaces. Possible formats are RGB, YUV 4:4:4, YUV 4:2:2, or YUV 4:2:0.

  ***Note*:** The video format is chosen at build time and cannot be changed at runtime.

- **Samples Per Clock:** Specifies the number of pixels processed per clock cycle. Permitted values are one, two, four, and eight samples per clock. This parameter determines the IP throughput. The more samples per clock, the larger throughput it provides. The larger throughput always needs more hardware resources.

  ***Note*:** This property applies to all layers that have a streaming interface.

- **Maximum Data Width:** Specifies the bit width of input and output samples on all the streaming interfaces. Permitted values are 8, 10, 12, and 16 bits.

  ***Note*:** This property applies to all layers that have a streaming interface.

- **Maximum Number of Columns:** Specifies maximum active video columns/pixels the IP core could produce at runtime. Any video width that is less than the Maximum Number of Columns can be programmed through AXI4-Lite control interface without regenerating the core.
- **Maximum Number of Rows:** Specifies maximum active video rows/lines the IP core could produce at runtime. Any video height that is less than Maximum Number of Rows can be programmed through the AXI4-Lite control interface without regenerating the core.
- **Address Width:** Specifies the address width of the AXI master interfaces for memory layers, either 32 or 64 bits.
- **Number of Overlay Layers:** Specifies the number of overlay layers with a minimum of one and a maximum of 16. When selecting this parameter as 0 in GUI, only main layer is active, the logo layer is enabled by default, and the Video Mixer is essentially a logo overlay function.
- **Memory Video Format:** Specifies the order in which the pixel component data is written into.

  ***Note*:** This option is available only for AMD Versal™ AI Edge Series Gen 2 devices. For all other devices, memory video format is fixed to raster order by default.

- **Enable CSC Coefficient Registers:** When selected, this parameter includes all the CSC coefficient registers which you can program.
- **Use URAM for internal FIFOs:** Select this check box to force the tool to use UltraRAM for implementation of internal FIFOs.
- **Use URAM for Line buffers:** Select this check box to force the tool to use UltraRAM for implementation of line buffers in Tile format. This option is available only for Versal AI Edge Series Gen 2 devices when Samples per clock = 8. For other devices or Sample per clock < 8, the tool uses either block RAM or UltraRAM based on the resource availability.

Layer Settings

The following settings apply to optional overlay layers with ID 1 through 16.

- **Layer ID:** Identifies the layer ID.

- **Video Format:** Specifies the video format per layer. Possible formats are RGB, YUV 4:4:4, YUV 4:2:2, or YUV 4:2:0, RGBA, and YUVA444 in case the layer interface type is streaming. If the layer interface type is memory mapped with raster scan order then the possible formats are: RGBX8, BGRX8, YUVX8, RGBA8, BGRA8, YUVA8, YUYV8, UYVY8, RGBX10, YUVX10, RGB565, Y_UV8, and Y_UV8_420, RGB8, BGR8, YUV8,Y_UV10, Y_UV10_420, Y8, Y10, Y_U_V8, Y_U_V10, Y_U_V12. If the layer interface type is memory mapped with tile scan order then the possible formats are: Y8, Y10, Y12, Y_UV8, Y_UV10, Y_UV12, Y_UV8_420, Y_UV10_420, Y_UV12_420, Y_U_V8, Y_U_V10, and Y_U_V12.

  ***Note*:** The video format is chosen at build time and cannot be changed at runtime.

- **Enable Global Alpha:** When selected, this enables alpha blending for this layer with the layer underneath. The alpha value needs to be programmed at runtime through the AXI4-Lite control interface.

  ***Note*:** If any of the per pixel alpha formats are selected for a layer, then the global alpha is automatically enabled.

- **Enable Scaling:** When selected, this enables scaling for this layer. The scale factor (1x, 2x, or 4x) needs to be programmed at runtime through the AXI4-Lite control interface.

- **Line Buffer Width:** This is only enabled when scaling is enabled for a layer and specifies the maximum width of this layer before scaling. A line buffer with this dimension is allocated in the block RAM to allow for scaling through line repeat. The width needs to be a multiple of the samples per clock setting.

  ***Note*:** The maximum supported value for LineBuffer width is upto 4096. By default, the set value is 1920. If scaling is enabled, LineBuffer width * scaling factor <= Min (4096, Max Number of Columns).

- **Interface Type:** Determines whether a layer is either a memory layer or a streaming layer. A memory layer has a memory mapped AXI4 interface while a streaming layer has an AXI4-Stream interface.

Logo Layer Settings

The following settings apply to the optional logo layer.

- **Enable Logo Layer:** When selected, this includes the logo layer. Block RAM is used for pixel storage of the logo according to the dimension settings. The logo is limited to eight bits per color component, and always needs to be formatted as RGB.

- **Maximum Number of Columns for Logo:** Specifies maximum pixel width of the logo. The width needs to be a multiple of the samples per clock setting. This setting affects the block RAM utilization.

- **Maximum Number of Rows for Logo:** Specifies maximum line height of the logo. This setting affects the block RAM utilization.

- **Enable Logo Transparency Color:** When selected, this allows for runtime programmability of a specified color key that becomes transparent.

- **Enable Logo Per Pixel Alpha:** When selected, this adds per pixel alpha blending functionality for the logo.

User Parameters

The following table shows the relationship between the fields in the Vivado IDE and the User Parameters (which can be viewed in the Tcl Console).

Table 15: **Vivado IDE Parameter to User Parameter Relationship**

Vivado IDE Parameter	User Parameter	Default Value
Top-Level Parameters		
Number of Video Components	NUM_VIDEO_COMPONENTS	3
Samples per Clock	SAMPLES_PER_CLOCK	1
Maximum Data Width	MAX_DATA_WIDTH	8
Maximum Number of Columns	MAX_COLS	3,840
Maximum Number of Rows	MAX_ROWS	2,160
AXIMM Data Width	AXIMM_DATA_WIDTH	64
AXIMM Address Width	AXIMM_ADDR_WIDTH	32
Number Read Outstanding	AXIMM_NUM_OUTSTANDING	4
Transaction Burst Length	AXIMM_BURST_LENGTH	16
Video Format	VIDEO_FORMAT	RGB
RGB	0	
YUV 4:4:4	1	
YUV 4:2:2	2	
YUV 4:2:0	3	
Number of Layers	NR_LAYERS	4
Memory Video Format	IS_TILE_FORMAT	Raster (0)
Use URAM for internal FIFOs	USE_URAM_FIFO	0
Use URAM for line buffers	USE_URAM_LINEBUF	0
Layer Parameters		
RGB	0	
YUV 4:4:4	1	
YUV 4:2:2	2	
YUV 4:2:0	3	
RGBA	5	
YUVA444	6	
RGBX8	10	
YUVX8	11	
YUYV8	12	
RGBA8	13	
YUVA8	14	

Table 15: **Vivado IDE Parameter to User Parameter Relationship** *(cont'd)*

Vivado IDE Parameter	User Parameter	Default Value
RGBX10	15	
YUVX10	16	
RGB565	17	
Y_UV8	18	
Y_UV8_420	19	
RGB8	20	
YUV8	21	
Y_UV10	22	
Y_UV10_420	23	
Y8	24	
Y10	25	
BGRA8	26	
BGRX8	27	
UYVY8	28	
BGR8	29	
Y_U_V8	42	
Y_U_V10	43	
Y_U_V12	44	
Layer i Enable Global Alpha	LAYERi_ALPHA	FALSE
Layer i Enable Scaling	LAYERi_UPSAMPLE	FALSE
Layer i Maximum Width	LAYERi_MAX_WIDTH	1,920
Layer i Interface Type	LAYERi_INTF_TYPE	Memory
Memory	0	
Stream	1	
Logo Parameters		
Enable Logo Layer	LOGO_LAYER	FALSE
Maximum Number of Columns for Logo	MAX_LOGO_COLS	64
Maximum Number of Rows for Logo	MAX_LOGO_ROWS	64
Enable Logo Transparency Color	LOGO_TRANSPARENCY_COLOR	FALSE
Enable Logo per Pixel Alpha	LOGO_PIXEL_ALPHA	FALSE
Enable CSC Coefficient Registers	ENABLE_CSC_COEFFICIENT_REGISTERS	FALSE

Output Generation

For details, see the *Vivado Design Suite User Guide: Designing with IP* (UG896).

Constraining the Core

This section contains information about constraining the core in the Vivado Design Suite.

Required Constraints

This section is not applicable for this IP core.

Device, Package, and Speed Grade Selections

This section is not applicable for this IP core.

Clock Frequencies

This section is not applicable for this IP core.

Clock Management

This section is not applicable for this IP core.

Clock Placement

This section is not applicable for this IP core.

Banking

This section is not applicable for this IP core.

Transceiver Placement

This section is not applicable for this IP core.

I/O Standard and Placement

This section is not applicable for this IP core.

Simulation

For comprehensive information about Vivado simulation components and information about using supported third-party tools, see the *Vivado Design Suite User Guide: Logic Simulation* (UG900).

IMPORTANT! *For cores targeting 7 series or Zynq 7000 devices, UNIFAST libraries are not supported. AMD IP is tested and qualified with UNISIM libraries only.*

Synthesis and Implementation

For details about synthesis and implementation, see the *Vivado Design Suite User Guide: Designing with IP* (UG896).

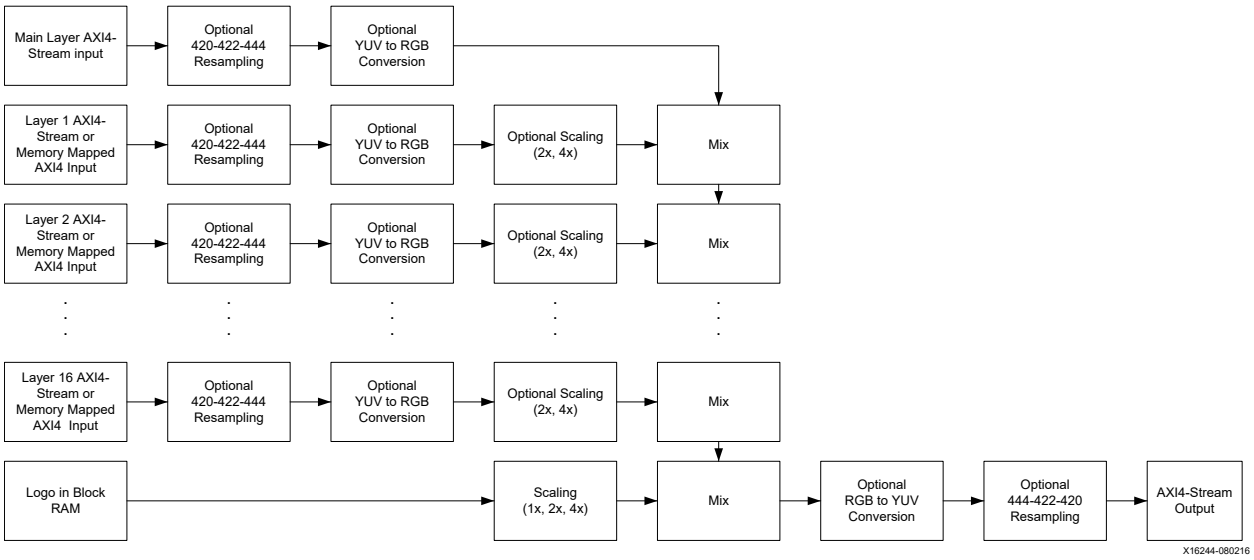
Designing with the Core

This chapter includes guidelines and additional information to facilitate designing with the core.

General Design Guidelines

The Video Mixer has support for up to 17 layers, with an optional logo layer, using a combination of video inputs from either frame buffer (through memory-mapped AXI4 interfaces) or streaming video cores (through AXI4-Stream interfaces). The following figure shows the functional block diagram of the Video Mixer. Functions listed as optional are under AMD Vivado™ Integrated Design Environment (IDE) control, as explained later in this section.

Figure 4: **Video Mixer Customize IP**



The Video Mixer always has one streaming input layer and one streaming output layer. This is known as main, master, or background layer. The main layer associates with the `s_axis_video` AXI4-Stream input and `m_axis_video` AXI4-Stream output.

The main layer input is the bottom-most layer and all other layers are blended on top. There is no programmable Z order for layers. The main layer can be disabled at runtime through the AXI4-Lite interface, by clearing Bit[0] of the Layer Enable (0x0040) Register. When disabled, the main layer does not read from the streaming interface but generates a solid background color as specified by Background Y or R (0x0028) Register, Background U or G (0x0030) Register, and Background V or B (0x0038) Register.

The video format of both the streaming input and output layer is determined by the Video Format field in the Vivado IDE. As video mixing is performed in the RGB domain, note that if the selected video format is YUV 4:4:4, color space conversion (as per the standard BT.601) is done at the input to RGB, and at the output to go back to YUV 4:4:4 again. If the selected video format is YUV 4:2:2 or YUV 4:2:0, additional chroma resampling at the input and output is performed to go from YUV 4:2:2 (or YUV 4:2:0) to YUV 4:4:4 and back.

The Video Mixer can have up to seventeen additional video or graphics sources, each of which can be configured to be an AXI4-Stream or memory mapped AXI4 frame buffer. When using streaming layers, keep in mind that the Video Mixer achieves synchronization of streaming layers to the main output layer by throttling the incoming data if the mixer is not ready to accept this data yet. Also, the mixer has no internal buffering to queue up incoming data.

If a streaming layer is used and enabled, the mixer tries to read from this streaming interface. If no data is present, the mixer stalls until data is present. Enable layers only when they start to receive video input on that layer. If the IP stalls and freezes, apply a hard reset to the core (as the core is still in stalled state regardless if you try to disable the layer).

If a memory mapped AXI4 frame buffer is selected to be a source, then the Video Mixer automatically handles interfacing to memory without the need of an additional AXI VDMA controller.

Like the main layer, every layer has a pre-defined video format that is either RGB, YUV 4:4:4, YUV 4:2:2, or YUV 4:2:0. Optional chroma resampling from YUV 4:2:2, or YUV 4:2:0 to YUV 4:4:4, and optional color space conversion from YUV 4:4:4 to RGB, are performed as mixing is done in the RGB domain. Layers 1 through 7, can also be configured to optionally have scaling ability (1x, 2x, or 4x) by the Enable Scaling field in the Vivado IDE. Scaling is implemented by means of pixel and line repeat, and therefore requires an internal line buffer.

The Video Mixer provides an optional logo layer. It blends a logo that is stored in the block RAM on the top-most layer. A programmable color key can be used to make part of the logo transparent. Also, (per pixel) alpha-blending can be used for logo transparency. The logo layer also has the ability for scaling (1x, 2x, or 4x). As the logo is read from the block RAM, no additional line buffer is needed for this. Scaling is implemented by means of pixel and line repeat.

Alpha Blending

Alpha blending is the process of combining two images with the appearance of partial transparency. To perform this composition, a per layer alpha value (and optionally a per pixel alpha value) is used that contains the coverage information for all the pixels within a layer. The alpha value ranges from 0 to 1, where 0 represents that the current pixel does not contribute to the final image and is fully transparent. A 1 represents that the current pixel is fully opaque. Any value in between represents a partially transparent pixel.

When applying alpha blending, the two pixels to be blended reside within two different image layers. Each layer has a definite Z-axis order. In other words, each layer resides closer or farther from the observer and has a different depth. Thus, the image pixel and the image pixel directly "over" it are to be blended.

The equation for alpha blending one layer to the layer directly behind in the Z-axis is below. This operation is conceptually simple linear interpolation between each color component of each layer.

$$\text{Component}'_{(x, y, z)} = \alpha\text{Component}_{(x, y, z)} + (1-\alpha)\text{Component}_{(x, y, z-1)}$$

Where:

α is the product of the global alpha and per pixel alpha (if enabled) or the global alpha value (otherwise).

$\text{Component}_{(x, y, z)}$ represents one color component channel from the color space triplet (RGB, YUV, etc.) associated with the pixel at coordinates (x, y) in Layer z.

$\text{Component}_{(x, y, z-1)}$ represents the same color component at the same (x, y) coordinates in Layer z – 1 (one layer below in Z-plane order).

$\text{Component}'_{(x, y, z)}$ is the resulting output component value after alpha-blending the component values from coordinates (x, y) from Layer z and Layer z – 1.

The same equation applies for the next layer above, Layer z + 1. These alpha-blending operations can be chained together by taking the resultant output, $\text{Component}'_{(x, y, z)}$, and substituting it into the Layer z + 1 equation for $\text{Component}_{(x, y, z)}$. This implies that the result of blending Layer z with the background becomes the new background for Layer z + 1, or the layer directly over it. In this mode, any number of image layers can be blended by taking the blended result of the layer below it.

***Note*:** Alpha blending is optional per layer but always enabled for the logo layer. In case alpha blending is disabled, pixels are always superimposed as fully opaque on the underlying layer.

Clocking

The Video Mixer has only one clock domain. All interfaces, that is, controller and target AXI4-Stream video interfaces as well as the AXI4-Lite interface, and also the memory mapped AXI4 interfaces uses the `ap_clk` pin as its clock source.

Pixel throughput of the Video Mixer core is defined by the product of the clock frequency times the Samples per Clock setting in the Vivado IDE. With a clock frequency of 300 MHz for `ap_clk`, and a two sample per clock configuration, the Video Mixer is capable of a 600 mega pixel throughput rate, which is sufficient to handle 4K resolutions at 60 Hz.

Resets

The Video Mixer has only a hardware reset option, `ap_rst_n` pin. No software reset option is available. The external reset pulse needs to be held for 16 or more `ap_clk` cycles to reset the core. The `ap_rst_n` signal is synchronous to the `ap_clk` clock domain. The `ap_rst_n` signal resets the entire core including the AXI4-Lite, AXI4-Stream, and memory mapped AXI4 interfaces.

System Considerations

The Video Mixer must be configured for the actual input and output image frame-size to operate properly. To gather the frame size information from the image video stream, it can be connected to the Video In to AXI4-Stream input and the Video Timing Controller. The timing detector logic in the Video Timing Controller gathers the video timing signals. The AXI4-Lite control interface on the Video Timing Controller allows the system processor to read out the measured frame dimensions, and program all downstream cores, such as the Video Mixer, with the appropriate image dimensions.

Another system consideration that needs to be taken into account is that there is sufficient bandwidth available to the Video Mixer to maintain proper functioning memory layers. The bandwidth needed (in MB/s) for a memory layer can be calculated with the following equation:

$$\text{Bandwidth (MB/s)} = \text{fps} \times \text{height} \times \text{stride}$$

Where `fps` is the number of frames per second the Video Mixer is operating at, `height` is the height in lines of the layer, and `stride` is the stride in bytes of the layer.

Programming Sequence

Most Video Mixer processing parameters other than image sizes can be changed dynamically and the change is picked up immediately. If the image size needs to be changed or the entire system needs to be restarted, it is recommended that pipelined AMD IP video cores are disabled/reset from system output towards the system input, and programmed/enabled from system output to system input.

Example Design

This chapter provides two example systems that include the Video Mixer. One is a simulation example design and the other one is a synthesizable example design. Important system-level aspects when designing with the Video Mixer are highlighted in these example designs, including:

- Video Mixer usage with memory mapped AXI4 interface memory layers.
- Typical usage of the Video Mixer with other cores.
- Video Mixer usage with AXI4-Stream interface layers (streaming input comes from the Video Frame Buffer Read IP).
- Runtime configuration of the Video Mixer by programming registers on-the-fly.

The supported platforms are listed in the following table.

Table 16: **Supported Platforms**

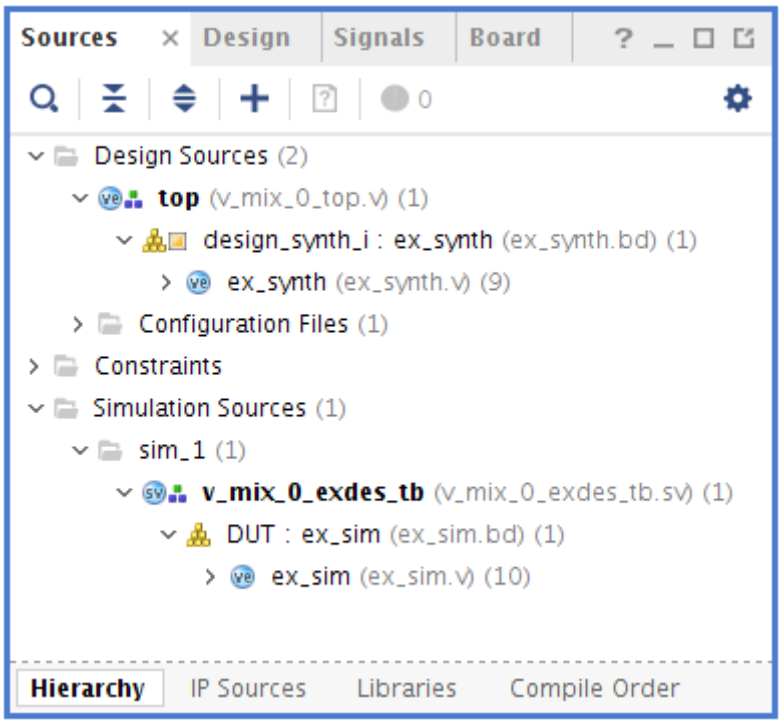
Development Boards	Additional Hardware	Processor
KC705	N/A	MicroBlaze™
ZCU102	N/A	psu_cortexa53_0
ZCU104	N/A	psu_cortexa53_0
ZCU106	N/A	psu_cortexa53_0
VCK190	N/A	CIPS

To open the example project, perform the following:

1. Select the **Video Mixer IP** from the AMD Vivado™ IP catalog.
2. Double-click the selected IP or right-click the IP and select **Customize IP** from the menu.
3. Configure the build-time parameters in the Customize IP window and click **OK**. The Vivado IDE generates an example design matching the build-time configuration.
4. In the Generate Output Products window, select **Generate** or **Skip**. If **Generate** is selected, the IP output products are generated after a brief moment.
5. Right-click **Video Mixer** in Sources panel and select **Open IP Example Design** from the menu.
6. In the Open IP Example Design window, select example project directory and click **OK**. The Vivado software then runs automation to generate the example design in the selected directory.

The generated project contains two example designs. The following figure shows the Source panel of the example project. Synthesizable example block design, along with top-level file, resides in Design Sources catalog. A corresponding constraint file is also provided for the synthesizable example design. Simulation example design files (including block design file, SystemVerilog test bench and another task file) are under Simulation Sources.

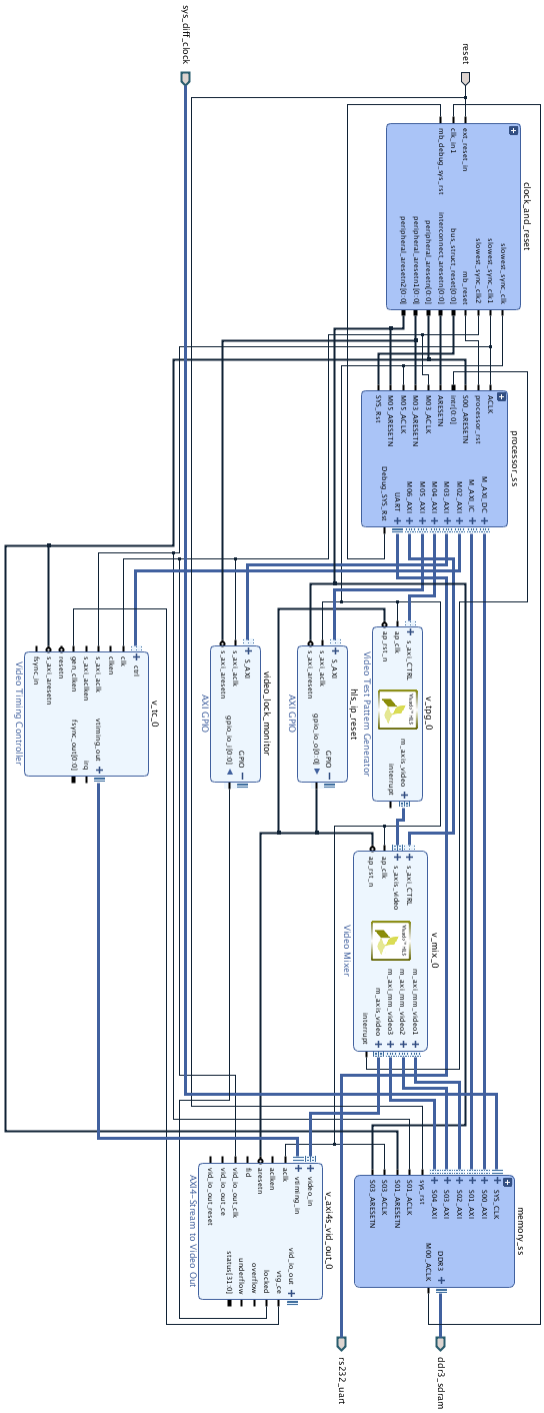
Figure 5: **Video Mixer Example Project Source Panel**



Synthesizable Example Design

The difference between the Synthesizable design and the Simulation example design is the use of a microprocessor instead of the AXI VIP core as AXI4 master. In addition, the synthesizable design uses the MIG IP core for DDR memory access. The `locked` port of AXI4-Stream to Video Out is connected to `axi_gpio_lock` core and the processor polls the corresponding register for a sign that the test passed. The following figure shows a synthesizable example design.

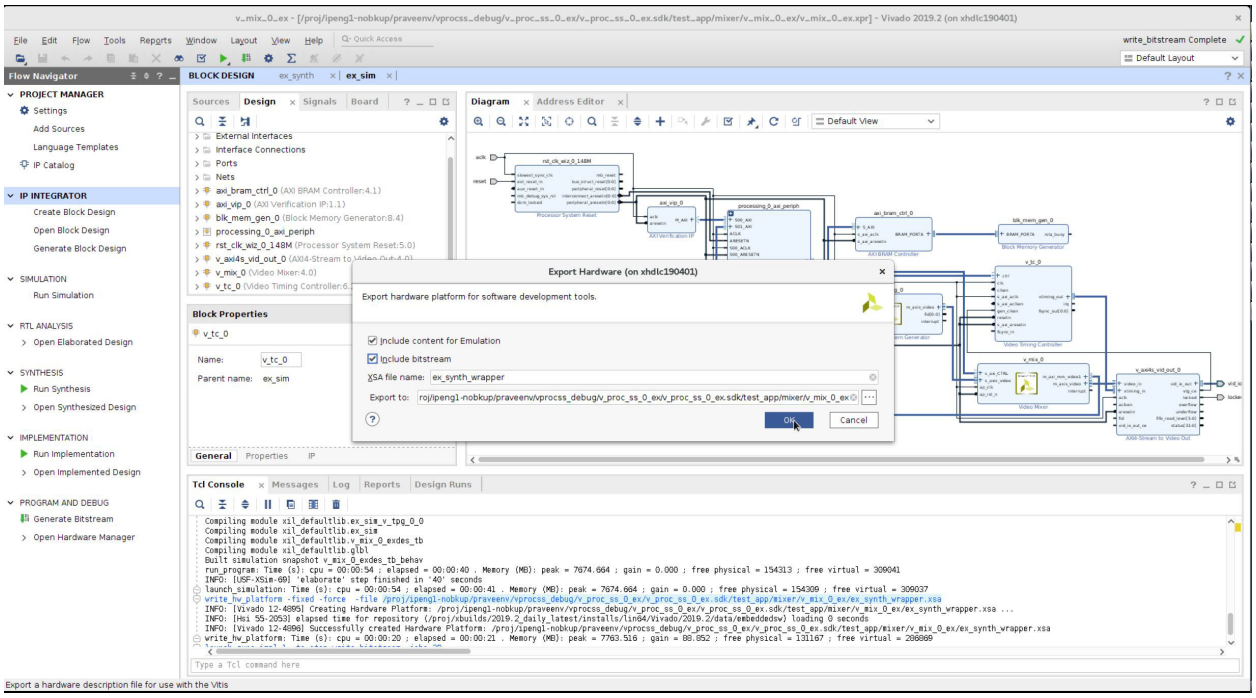
Figure 6: **Synthesizable Example Block Design**



The synthesizable example design requires both Vivado and AMD Vitis™ tools.

The first step is to run synthesis, implementation, and bitstream generation in Vivado. After all those steps are done, select **File → Export → Export Hardware**. In the window, select **Include bitstream**, select an export directory and click **OK** to create an XSA project.

Figure 7: **Export Hardware**



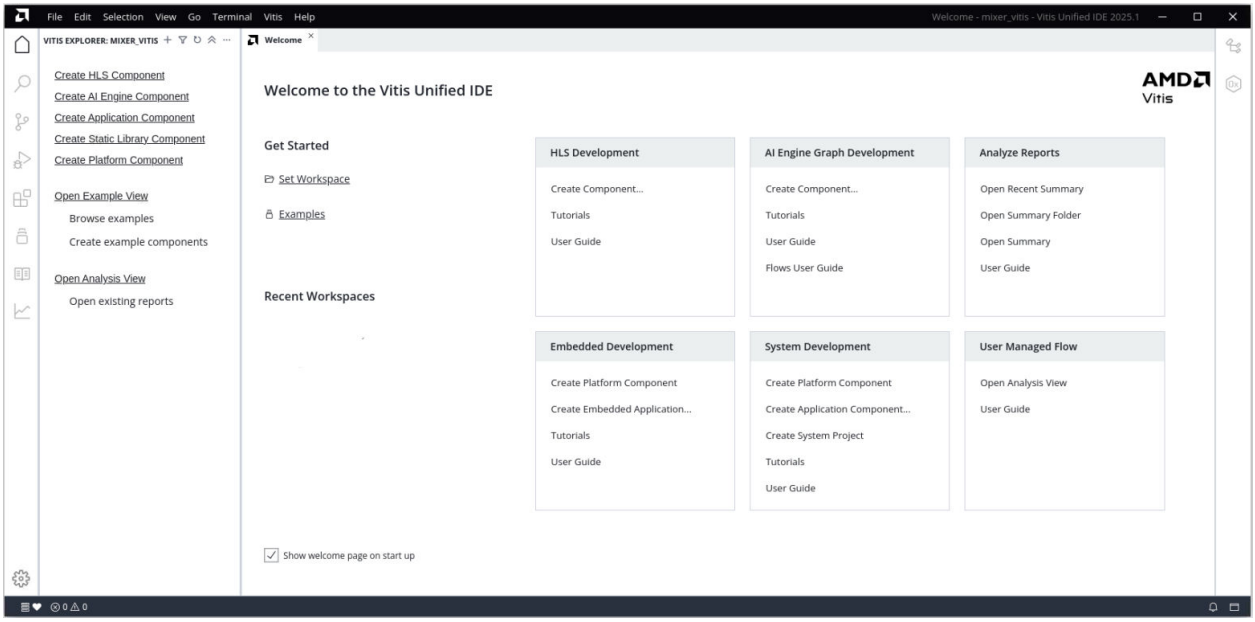
The remaining work is performed in the AMD Vitis tool. The Video Mixer example design file can be found in the following Vitis directory:

`(<install_directory>/<release>/data/embeddedsw/XilinxProcessorIPLib/drivers/v_mix_v6_1/examples/`

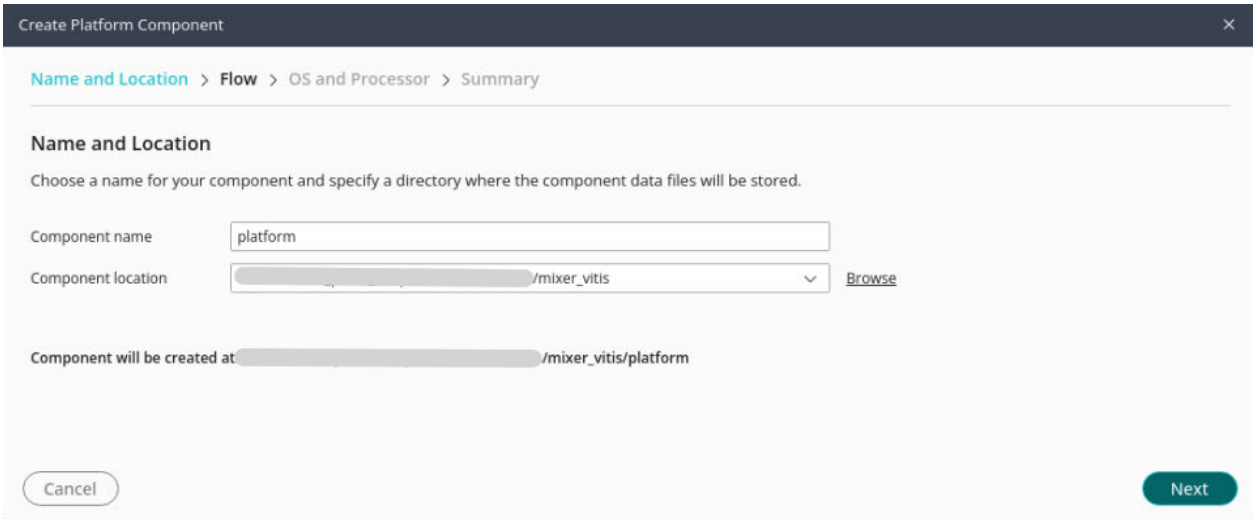
The example application design source files (contained within `examples` folder) are tightly coupled with the `v_mix` example design available in Vivado IP catalog.

Perform the following steps to get the .elf file from the Vitis application.

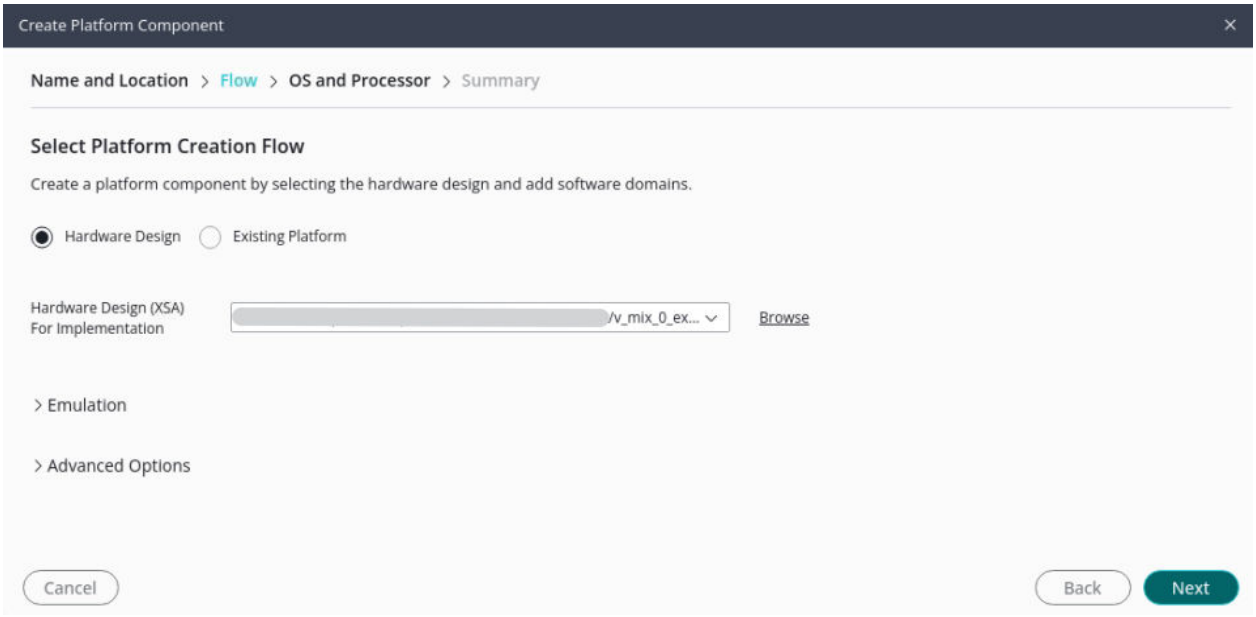
1. Open the Vitis application. Set workspace path by selecting **Get Started → Set Workspace**.



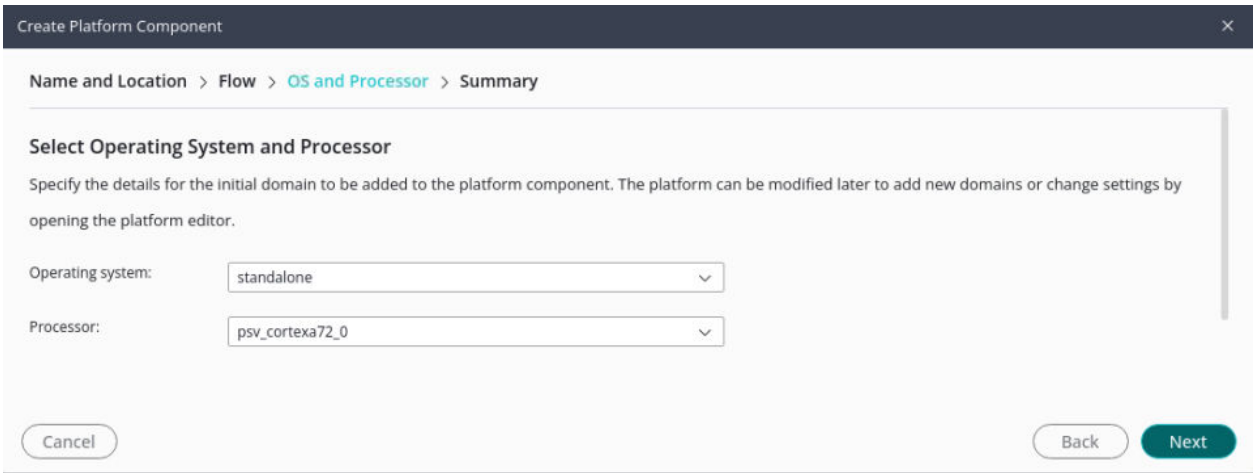
2. Create the platform component by selecting **Embedded Development → Create Platform Component**.
3. Enter the **Component name** and **Component location** in the pop-up window and click **Next**.



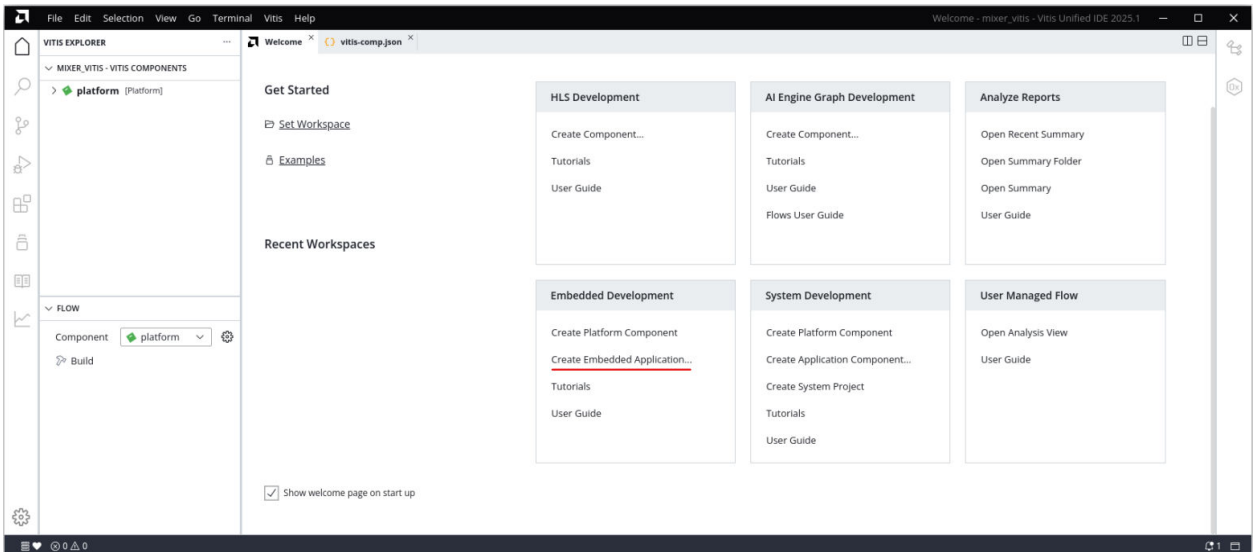
4. Select the required XSA.



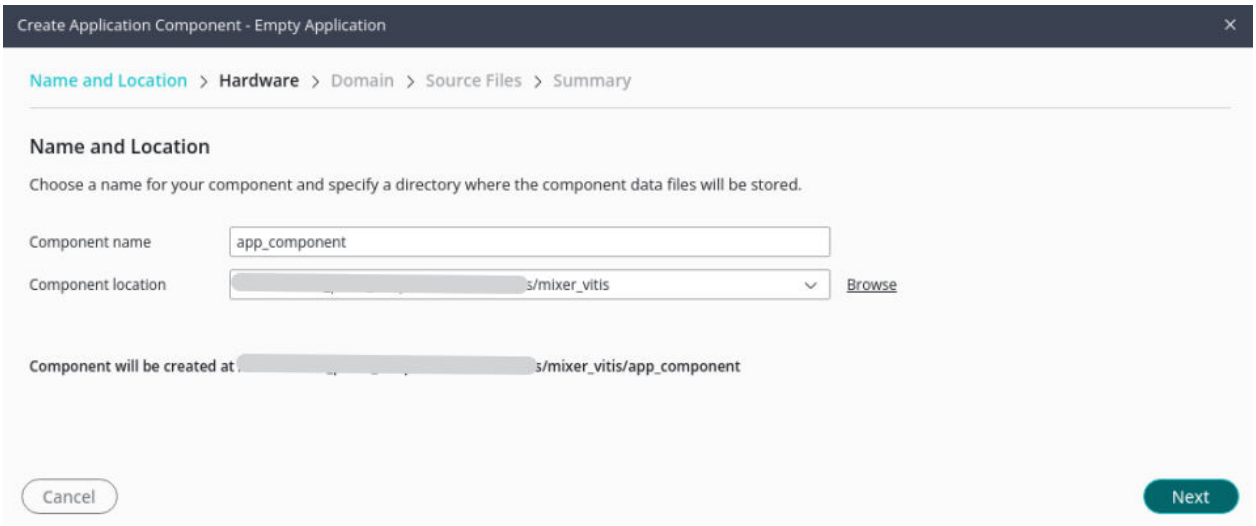
5. Select the **Operating System**, **Processor**, and **Compiler**.



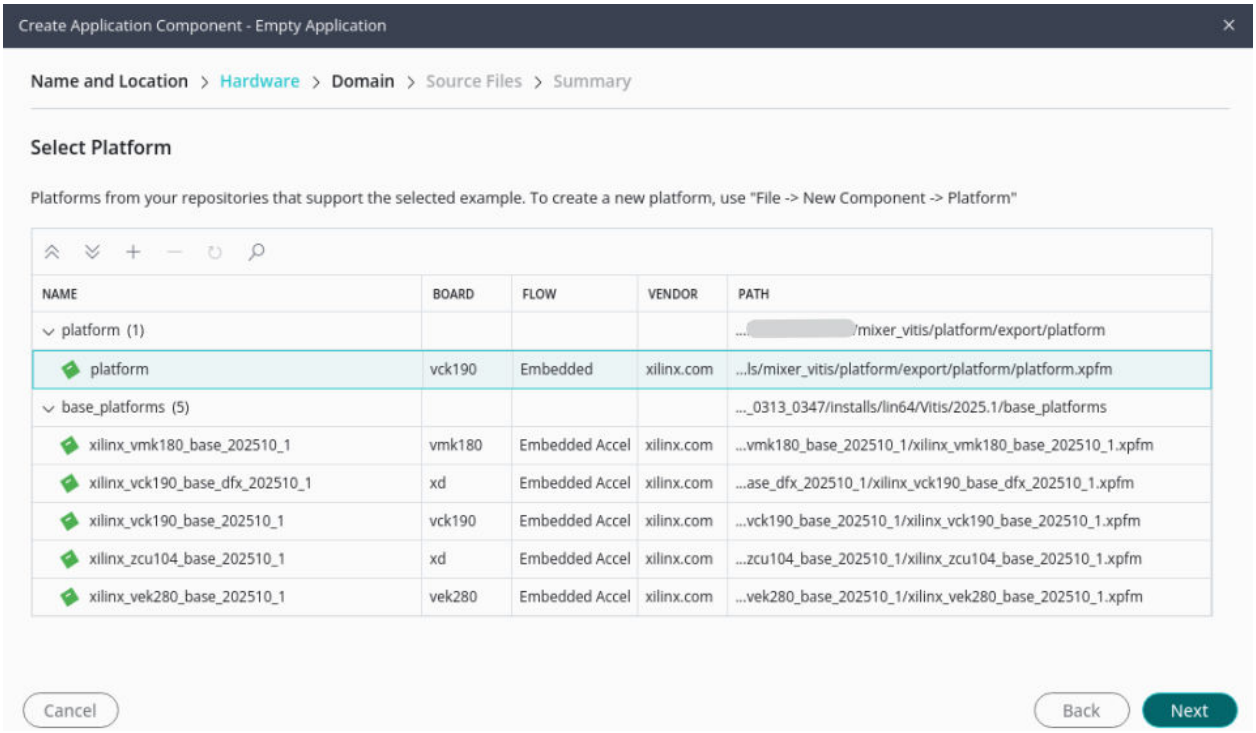
6. Review the summary and click **Finish**.
7. Once the platform component is created, switch to the welcome panel, and select the **Create Empty Embedded Application** option in **Embedded Development** → **Create Platform Component**.



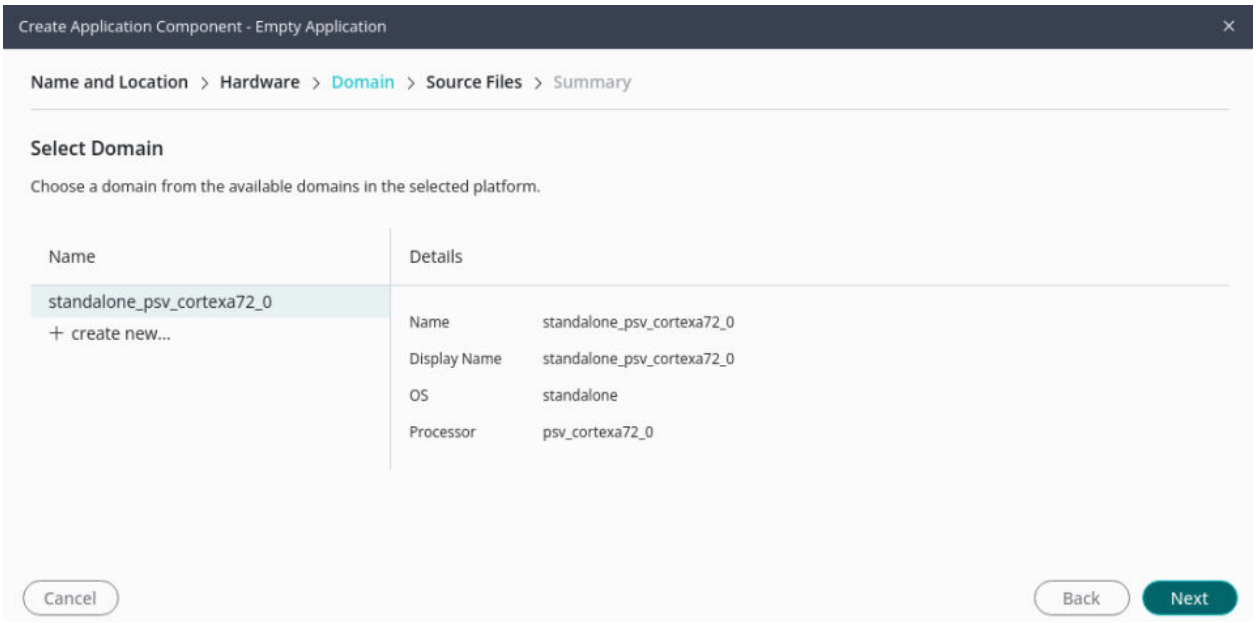
8. Choose the **Component name** and **Component location** in the pop-up window. Select **Next**.



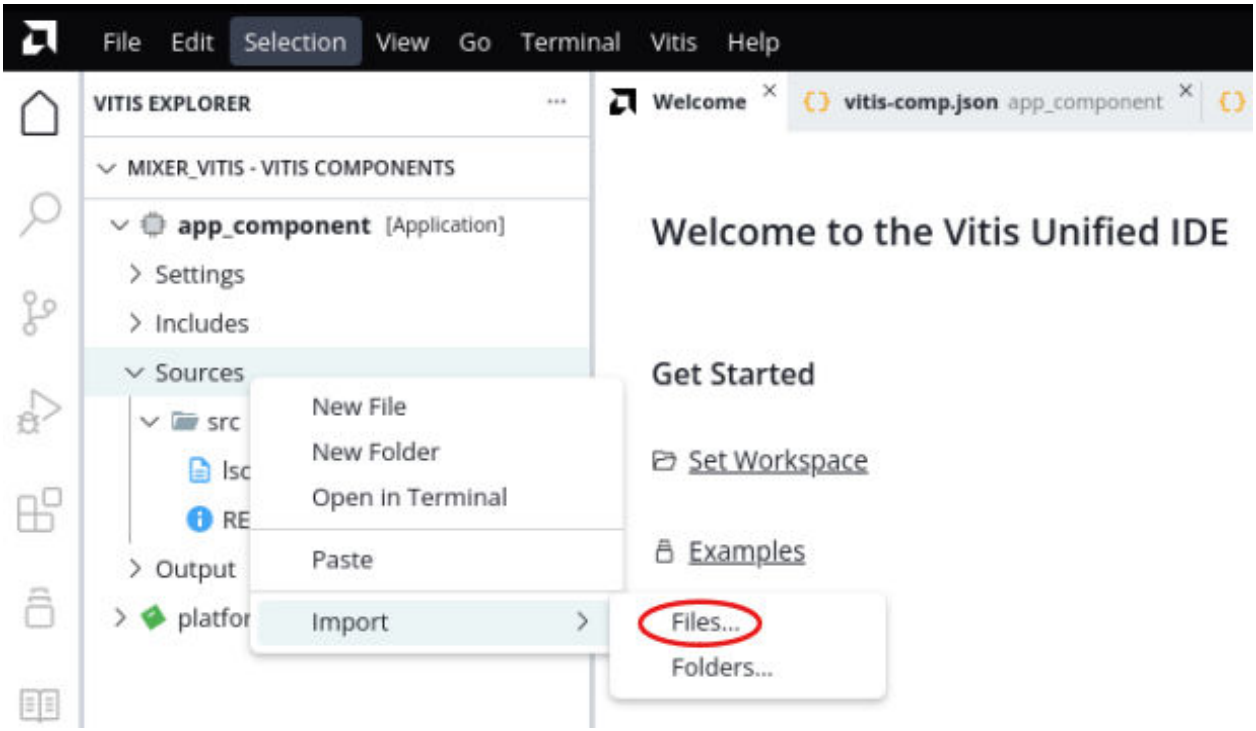
9. Select the required Platform in the hardware tab and click **Next**.



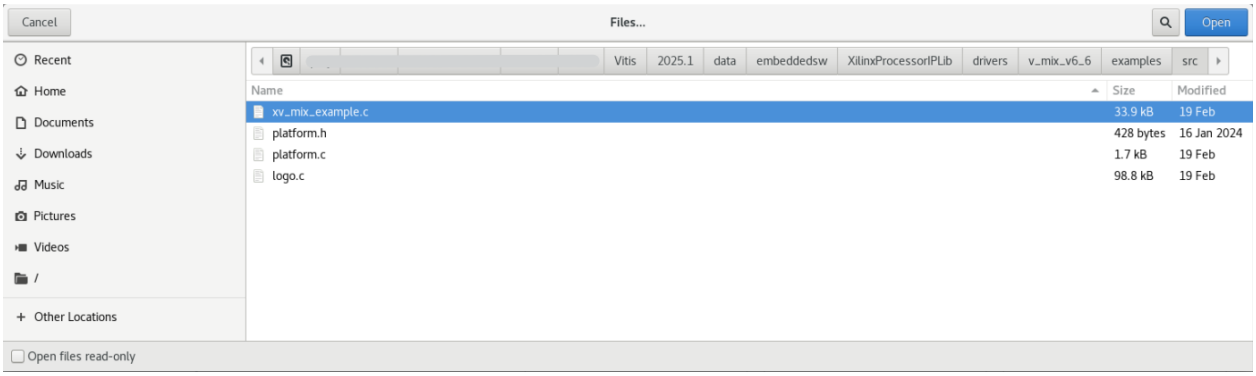
10. Choose the domain from the available domains.



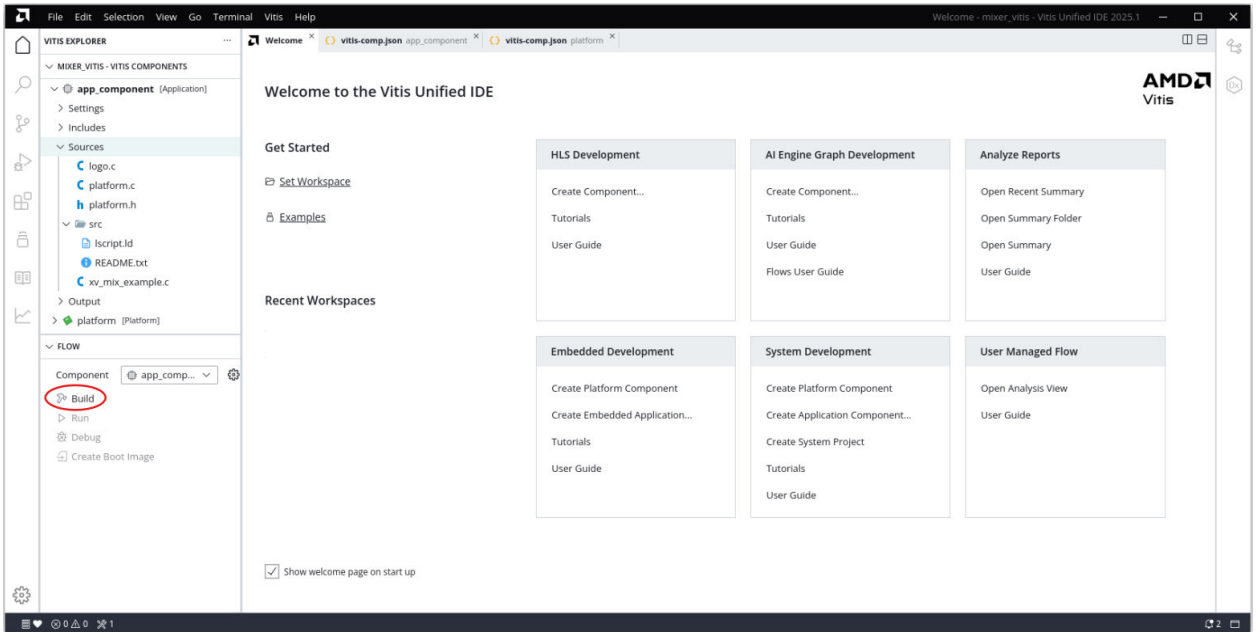
11. Add source files or skip for now, review the summary, and click **Finish**.
12. If skipped in the previous step, import the source files into the app component by right clicking on sources and selecting **Import → Files**.



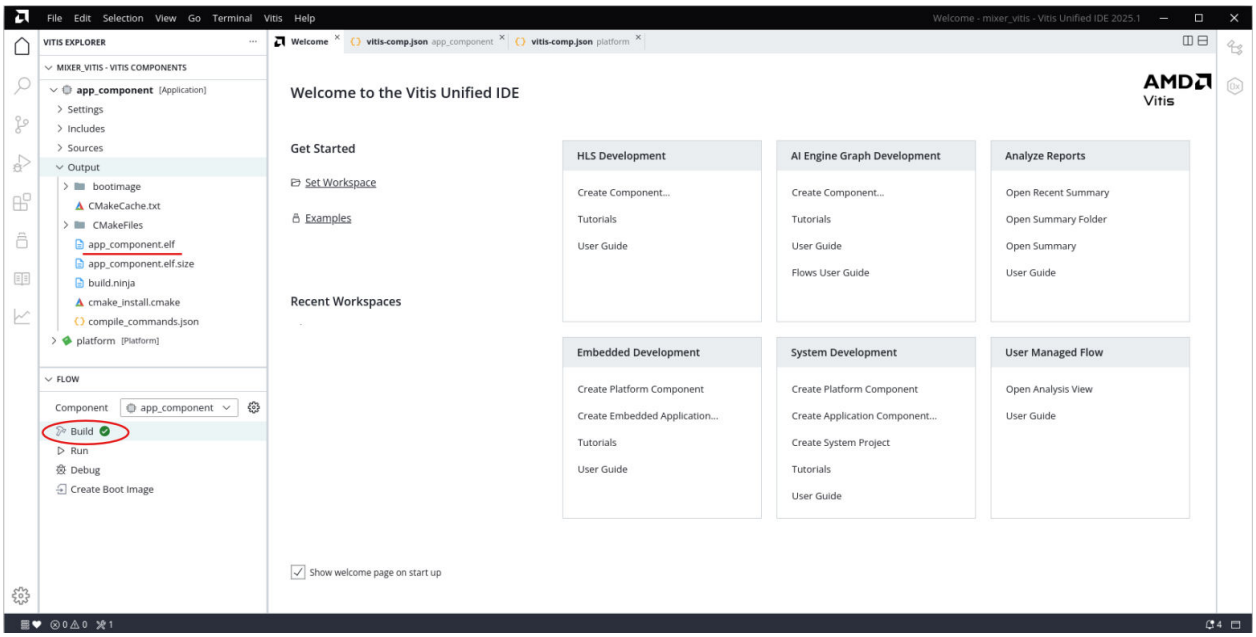
13. Import the required files.



14. Build the project by selecting **Flow → Build**. When prompted for Platform build, select **Yes**.



15. Once the build is complete, check the Output folder for the elf file.



The `vmix_example.tcl` automates the process of generating the downloadable bit and `elf` files from the provided example `xsa` file.

To run the provided Tcl script:

1. Copy the exported example design `xsa` file in the `examples` directory of the driver
2. Launch the Xilinx Software Command-Line Tool (xsct) terminal
3. `cd` into the examples directory
4. Source the `tcl` file `xsct`:

   ```
   %>source vmix_example.tcl
   ```

5. Execute the script:

   ```
   xsct%>vmix_example xsa_file_name.xsa
   ```

The Tcl script performs the following:

- Create workspace
- Create HW project
- Create BSP
- Create Application Project
- Build BSP and Application Project

After the process is complete, the required files are available in:

```
bit file ->v_mix_0_ex/ folder
```

```
elf file -> v_mix_0_ex/sdk/xv_mix_example_1{Debug/Release} folder
```

Next, perform the following steps to run the software application:

IMPORTANT! *To do so, make sure that the hardware is powered on and a Digilent Cable or an USB Platform Cable is connected to the host PC. Also, ensure that a USB cable is connected to the UART port of the board.*

1. Launch the Vitis application.
2. Set workspace to `vmix_example.sdk` folder in prompted window. The SDK project opens automatically (if a welcome page shows up, close that page).
3. Download the bitstream into the FPGA by selecting **Xilinx Tools → Program FPGA**. The Program FPGA dialog box opens.
4. Ensure that the Bitstream field shows the bitstream file generated by Tcl script, and then click **Program**.

   ***Note*:** The DONE LED on the board turns green if the programming is successful.
5. A terminal program (Hyper Terminal or PuTTY) is needed for UART communication. Open the program, choose appropriate port, set baud rate to 115,200, and establish Serial port connection.
6. Select and right-click the application `vmix_example_design` in Project_Explorer panel.
7. Select **Run As → Launch on Hardware (GDB)**.
8. Select **Binaries and Qualifier** in window and click **OK**.

The example design test result are shown in terminal program.

Appendix A

Verification, Compliance, and Interoperability

This appendix provides details about how this IP core was tested for compliance with the protocol to which it was designed.

Simulation

A highly parameterizable test bench was used to test the Video Mixer in AMD Vitis™ High-Level Synthesis (HLS). Testing included the following:

- Register accesses
- Processing multiple frames of data
- Varying IP throughput and pixel data width
- Testing the Video Mixer with AXI4-Stream and memory mapped AXI4 interface layers
- Testing of various frame sizes
- Varying parameter settings

Hardware Testing

The Video Mixer core has been validated at AMD to represent many different parameterizations. A test design was developed for the core that incorporated a processor, AXI4-Lite interconnect, and various other peripherals. The processor was responsible for:

- Programing the video clock to match tested video resolution
- Configuring the video IP cores with different resolutions
- Launching the test
- Reporting the Pass/Fail status of the test and any errors that were found

Interoperability

The core slave (input) and master (output) AXI4-Stream interface can work directly with any core that produces RGB, YUV 4:4:4, YUV 4:2:2, or YUV 4:2:0 video data.

Appendix B

Upgrading

This appendix contains information about upgrading to a more recent version of the IP core.

Upgrading in the Vivado Design Suite

This section is not applicable for the first release of the core.

Changes from v5.2 to v6.0

Video Mix v6.0 is a direct replacement for v5.2. In v6.0, the IP is provided with support for Tile format and UltraRAM resource use.

Changes from v5.1 to 5.2

Video Mix v5.2 is a direct replacement for v5.1. In v5.2, the IP is provided with the Versal Example Design.

Changes from v5.0 to 5.1

Video Mix v5.1 is a direct replacement for v5.0. The IP features remain the same but AMD Vitis™ HLS tool synthesizes the IP instead of AMD Vivado™ HLS tool.

Changes from v4.0 to 5.0

Added Enable CSC Coefficient Registers parameters in the GUI.

Changes from v3.0 to 4.0

The IP now supports up to 16 (was 8) overlay layers.

New bits are added to Layer enable register to support up to 16 overlay layers.

Logo layer enable bit changed from bit 15 to bit 23.

Offset addresses of the logo layer registers changed.

Changes from v2.0 to v3.0

The IP now supports up to 8 (was 7) overlay layers.

The layer enable bit for the logo layer has changed from bit 8 to bit 15 in register with offset 0x0040. Bit 8 now enables overlay layer 8.

A new memory video format has been added: BGR8.

Parameter Changes

There is one new parameter added:

USE_URAM. In AMD UltraScale+™ devices, line buffers can be stored in UltraRAM instead of Block RAM.

Port Changes

There are no port changes.

Other Changes

There are no other changes.

Changes from v1.0 to v2.0

Register offsets changed so that each layer is grouped together rather than grouping layer properties together. Two buffers are provided for semi-planar formats. In v1.0 of the IP, the chroma buffer address was automatically set based on frame size and stride. In v2.0 of the IP, the chroma buffer address must be specified through a register.

Two new streaming video formats, RGB with Alpha and YUV 4:4:4 with Alpha, are now available for use with the memory formats RGBA8, BGRA8, and YUVA8.

New memory video formats added: BGRX8 and UYVY8.

Both 32-bit and 64-bit addressing are supported for the memory interfaces.

Parameter Changes

A new parameter was added: `AXIMM_ADDR_WIDTH`. Both 32-bit and 64-bit addressing are supported for the memory interfaces.

Port Changes

There are no port changes.

Other Changes

There are no other changes.

Appendix C

Application Software Development

The Video Mixer core is delivered with a bare-metal driver as part of the AMD Vitis™ installation. The driver follows a layered architecture wherein layer 1 provides basic register peek/poke capabilities and requires you to be familiar with the register map and inner workings of the core. Layer 2, on the other hand, abstracts away all the lower level details and provides an easy to use functional interface to the Video Mixer. AMD recommends always using layer 2 APIs to interact with the core.

Building the BSP

When the Board Support Package (BSP) is built, the Video Mixer driver inherently pulls in the required dependency, that is, the video common driver. This driver contains the enumerations for video specific information like color format, color depth, frame rate, etc. It also defines fundamental data types to represent the video stream, timing and window that are used in the Video Mixer driver and is common across all AMD video drivers.

During the build process the Video Mixer driver extracts the Video Mixer hardware configuration settings from the provided hardware design file.

Prerequisites

If optional layers are enabled and configured as "Memory", there are certain requirements that must be taken care of while programming the core.

1. The core itself does not have a data realignment engine and therefore the application software must align the layer memory addresses before writing to the registers. The alignment requirement is specified below, and makes sure that the start address is aligned with the width of the memory interface.

   $$2 \times \text{Pixels per Clock} \times 4 \text{ Bytes}$$

2. The Stride value (in bytes) must be aligned as above to make sure that every row of pixels starts at an aligned memory location. Use the following equation to compute the stride from the width (in pixels) in raster-scan order:

$$\text{Stride in Bytes} \geq (\text{Width} \times \text{Bytes per Pixel})$$

The bytes per pixel value varies per video in memory format, and is described in Memory Mapped AXI4 Interface.

Use the following equation to compute the stride from the width (in pixels) in tile-scan order for a tile size of n × 4 (where n = 32 or 64):

$$\text{Stride in Bytes} \geq (((\text{Width} + (n-1)) \,\&\, \sim(n-1)) * 4 * \text{bit-depth}) / 8$$

Note: Due to a current limitation of the IP core when configured for samples per clock=8, the minimum stride required for tiled video format with bit-depth=10 needs to be calculated using n=64 in the above formula. This calculation applies to both 32x4 and 64x4 tiles.

Padding bytes is sometimes necessary (hence, the ≥ in the equation) to make sure that every row of pixels starts at an address that is aligned with the size of the data on the memory mapped interface as described in Layer Stride (0x0#20) Register.

3. Layer Window Start Horizontal Position and Width must be a multiple of Pixels per Clock as selected in the Vivado Integrated Design Environment (IDE) for this core.
4. In Tile video format, the Width value must be a multiple of 4 and Pixels per Clock. The Height value must be a multiple of 8.

Modes of Operation

The Video Mixer supports two modes of operation, which require two different programming models.

- **Auto Restart Mode (Default):** The driver initialization routine configures the Video Mixer for auto restart mode. In this mode, after the current frame is processed the core automatically triggers the start of the next frame processing. Consequently, the core can keep on processing frames without any software intervention, with settings applied when the core was started. This mode is used when there is no need to make frame synchronous changes to the core registers. This is the case if streaming layers are being used.

  You can switch to Auto Restart Mode, at any time, by disabling the interrupts, using the `XVMix_InterruptDisable` API.

- **Interrupt Mode:** When using memory layers, frame synchronous changes to the layer memory buffer address are key for the correct operation of the Video Mixer, that is, reading from memory needs to be synchronized with writing to memory by for example a graphics engine. In this case, the interrupt mode should be used. In this mode, the interrupt (IRQ) port of the core needs to be connected to a system interrupt controller.

When an interrupt is triggered, the core interrupt service routine (ISR) checks to confirm if current frame processing is complete. It then calls a user programmable callback function, if a callback function has been registered. In the callback function, the register settings for the next frame should be programmed, that is, what layer buffer address a layer should read next from. Finally, the interrupt service routine triggers the core to start processing the next frame.

You can switch to Auto Restart Mode, at any time, by disabling the interrupts, using the `XVMix_InterruptDisable` API.

An application must perform the following tasks to configure the core for Interrupt mode.

- Register the core ISR routine `XVMix_InterruptHandler` with the system interrupt controller.
- Register the application callback function that should be called within the interrupt context. This can be done using the API `XVMix_SetCallback`.
- Enable the interrupts by calling provided API `XVMix_InterruptEnable`.

Usage

To better understand the driver usage, consider the following test case scenario. Suppose the core in the design was configured with few memory layers and each layer has certain optional features, like alpha blending and/or scaling enabled, and that the logo layer is enabled with the optional color key feature. Because memory layers are being used, there are sources in the design that generates frame data for each of the Video Mixer memory layers. The application should allocate required frame buffer space per layer in memory. These addresses should be updated during the interrupt.

To integrate and use the Video Mixer driver in the application, the following steps should be followed:

1. Include the driver header file `xv_mix_l2.h` that contains the mixer instance object definition.
2. Declare an instance of the Video Mixer type: `XV_Mix_l2 Mixer;`
3. Initialize the Video Mixer instance at power on:

   ```
   int XVMix_Initialize(XV_Mix_l2 *InstancePtr, u16 DeviceId);
   ```

   This function accesses the hardware configuration and initializes the core to the power on default state.

   - Set Master layer to 1080p.
   - Set Background color to blue.
   - Enable the master layer.

- Set the operating mode to Auto Restart.

4. If the core is operating in interrupt mode, the application needs to perform the tasks mentioned, that is, register the ISR with the system interrupt controller and set the application callback function. This function is called by the Video Mixer driver when the frame done IRQ is triggered.
5. If applicable, write the application level callback function. An example action to be performed here would be to update layer buffer addresses from where to read the next frame data for each layer. This allows the application to render the frame updates in memory, on screen.
6. Write a function to configure the core. The following is a sample event sequence that might be performed here:
   a. Set the master layer video stream properties:

      ```
      void XVMix_SetVidStream(XV_Mix_l2 *InstancePtr, const
      XVidC_VideoStream *StrmIn);
      ```

   b. For each memory layer, set the frame buffer base address:

      ```
      int XVMix_SetLayerBufferAddr(XV_Mix_l2 *InstancePtr,
      XVMix_LayerId LayerId, UINTPTR Addr);
      ```

   c. For semi-planar formats, set the buffer base address for the second plane:

      ```
      int XVMix_SetLayerChromaBufferAddr(XV_Mix_l2 *InstancePtr,
      XV_Mix_LayerId LayerId, UINTPTR Addr);
      ```

   d. If logo layer is enabled, load the logo data:

      ```
      int XVMix_LoadLogo(XV_Mix_l2 *InstancePtr, XVidC_VideoWindow
      *Win, u8 *RBuffer, u8 *GBuffer, u8 *BBuffer);
      ```

   e. If logo color key feature is enabled, set the default color key data:

      ```
      int XVMix_SetLogoColorKey(XV_Mix_l2 *InstancePtr,
      XVMix_LogoColorKey ColorKeyData);
      ```

   f. For each layer, set the window properties:

      ```
      int XVMix_SetLayerWindow(XV_Mix_l2 *InstancePtr, XVMix_LayerId
      LayerId, XVidC_VideoWindow *Win, u32 StrideInBytes);
      ```

   g. Enable the interrupts (if operating in interrupt mode):

      ```
      void XVMix_InterruptEnable(XV_Mix_l2 *InstancePtr);
      ```

   h. Enable the master layer (and additional memory layers if needed). (Enable streaming layers only when they start to receive video input on that layer.):

      ```
      int XVMix_LayerEnable(XV_Mix_l2 *InstancePtr, XVMix_LayerId
      LayerId);
      ```

   i. Finally, start the core:

```
void XVMix_Start(XV_Mix_l2 *InstancePtr);
```

7. If optional features like alpha blending, enable CSC coefficient registers or scaling are enabled for a given layer, then these can be updated using the provided APIs.

```
int XVMix_SetLayerAlpha(XV_Mix_l2 *InstancePtr, XVMix_LayerId
LayerId, u16 Alpha);

int XVMix_SetLayerScaleFactor(XV_Mix_l2 *InstancePtr, XVMix_LayerId
LayerId, XVMix_Scalefactor Scale);

int XVMix_SetLayerScaleFactor(XV_Mix_l2 *InstancePtr, XVMix_LayerId
LayerId, XVMix_Scalefactor Scale);

u32 XVMix_SetCscCoeffs(XV_Mix_l2 *InstancePtr,XVidC_ColorStd
colorStandard, XVidC_ColorRange colorRange, u8 colorDepth);
```

You are encouraged to experiment with other APIs that allow the layer window to be resized or moved on the screen or change logo color key information, if applicable. Each API provides a return value indicating if the desired action was successful or not.

Also, to help debug the Video Mixer, two Debug APIs are available that provide the state of the core:

```
void XVMix_DbgReportStatus(XV_Mix_l2 *InstancePtr);

void XVMix_DbgLayerInfo(XV_Mix_l2 *InstancePtr, XVMix_LayerId
LayerId);
```

Note:

1. Resolution changes are not possible on the fly. If either the master input stream resolution changes during operation or the output resolution needs to be changed, then the application must reset the Video Mixer, that is, toggle `ap_rst_n`, and reconfigure the core for the new resolution. After reset, all registers are cleared to 0.
2. Certain actions cannot be performed when the core is in middle of processing a frame. For example, if a window has to be resized or the scaling factor has to be changed for a layer, then this layer should be disabled first. Next, apply the new settings and lastly re-enable the layer. Alternatively, in interrupt mode these operations can be done in the user callback function registered with the mixer interrupt service routine.
3. When resizing, moving, or scaling a layer window, the driver checks to ensure that the window properties provided does not cause the new window to go out of frame boundary. If any of these actions cause such a condition, then the action cannot applied and the API returns with an error code. The application code should check the return status of all APIs to make sure the required action was completed successfully and if not take corrective action.
4. Do not enable any layer until you start receiving video input on that layer. If a layer is enabled before receiving the stream, the IP can freeze and then you must apply a hard reset on the IP. A hard reset is the only way to recover after the IP is in this frozen state.

AMD

Appendix D

Debugging

This appendix includes details about resources available on the AMD Support website and debugging tools.

Finding Help with AMD Adaptive Computing Solutions

To help in the design and debug process when using the core, the Support web page contains key resources such as product documentation, release notes, answer records, information about known issues, and links for obtaining further product support. The Community Forums are also available where members can learn, participate, share, and ask questions about AMD Adaptive Computing solutions.

Documentation

This product guide is the main document associated with the core. This guide, along with documentation related to all products that aid in the design process, can be found on the AMD Adaptive Support web page or by using the AMD Adaptive Computing Documentation Navigator. Download the Documentation Navigator from the Downloads page. For more information about this tool and the features available, open the online help after installation.

Answer Records

Answer Records include information about commonly encountered problems, helpful information on how to resolve these problems, and any known issues with an AMD Adaptive Computing product. Answer Records are created and maintained daily to ensure that users have access to the most accurate information available.

Answer Records for this core can be located by using the Search Support box on the main AMD Adaptive Support web page. To maximize your search results, use keywords such as:

- Product name
- Tool message(s)

- Summary of the issue encountered

A filter search is available after results are returned to further target the results.

Master Answer Record for the Core

AR 66753.

Technical Support

AMD Adaptive Computing provides technical support on the Community Forums for this AMD LogiCORE™ IP product when used as described in the product documentation. AMD Adaptive Computing cannot guarantee timing, functionality, or support if you do any of the following:

- Implement the solution in devices that are not defined in the documentation.
- Customize the solution beyond that allowed in the product documentation.
- Change any section of the design labeled DO NOT MODIFY.

To ask questions, navigate to the Community Forums.

Debug Tools

There are many tools available to address Video Mixer design issues. It is important to know which tools are useful for debugging various situations.

Vivado Design Suite Debug Feature

The AMD Vivado™ Design Suite debug feature inserts logic analyzer and virtual I/O cores directly into your design. The debug feature also allows you to set trigger conditions to capture application and integrated block port signals in hardware. Captured signals can then be analyzed. This feature in the Vivado IDE is used for logic debugging and validation of a design running in AMD devices.

The Vivado logic analyzer is used to interact with the logic debug LogiCORE IP cores, including:

- ILA 2.0 (and later versions)
- VIO 2.0 (and later versions)

See the *Vivado Design Suite User Guide: Programming and Debugging* (UG908).

Hardware Debug

Hardware issues can range from link bring-up to problems seen after hours of testing. This section provides debug steps for common issues. The Vivado debug feature is a valuable resource to use in hardware debug. The signal names mentioned in the following individual sections can be probed using the debug feature for debugging the specific problems.

General Checks

Ensure that all the timing constraints for the core were properly incorporated from the example design and that all constraints were met during implementation.

- Does it work in post-place and route timing simulation? If problems are seen in hardware but not in timing simulation, this could indicate a PCB issue. Ensure that all clock sources are active and clean.
- If using MMCMs in the design, ensure that all MMCMs have obtained lock by monitoring the `locked` port.

Appendix E

Additional Resources and Legal Notices

Finding Additional Documentation

Technical Information Portal

The AMD Technical Information Portal is an online tool that provides robust search and navigation for documentation using your web browser. To access the Technical Information Portal, go to https://docs.amd.com.

Documentation Navigator

Documentation Navigator (DocNav) is an installed tool that provides access to AMD Adaptive Computing documents, videos, and support resources, which you can filter and search to find information. To open DocNav:

- From the AMD Vivado™ IDE, select **Help → Documentation and Tutorials**.
- On Windows, click the **Start** button and select **Xilinx Design Tools → DocNav**.
- At the Linux command prompt, enter `docnav`.

***Note*:** For more information on DocNav, refer to the *Documentation Navigator User Guide* (UG968).

Design Hubs

AMD Design Hubs provide links to documentation organized by design tasks and other topics, which you can use to learn key concepts and address frequently asked questions. To access the Design Hubs:

- In DocNav, click the **Design Hubs View** tab.
- Go to the Design Hubs web page.

Support Resources

For support resources such as Answers, Documentation, Downloads, and Forums, see Support.

References

These documents provide supplemental material useful with this guide:

1. *Vivado Design Suite User Guide: Release Notes, Installation, and Licensing* (UG973)
2. *Vivado Design Suite: AXI Reference Guide* (UG1037)
3. *AXI4-Stream Video IP and System Design Guide* (UG934)
4. *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* (UG994)
5. *Vivado Design Suite User Guide: Designing with IP* (UG896)
6. *Vivado Design Suite User Guide: Getting Started* (UG910)
7. *Vivado Design Suite User Guide: Logic Simulation* (UG900)
8. *ISE to Vivado Design Suite Migration Guide* (UG911)
9. *Vivado Design Suite User Guide: Programming and Debugging* (UG908)
10. *Vivado Design Suite User Guide: Implementation* (UG904)
11. *Video Processing Subsystem Reference Design* (XAPP1291)
12. *Video Processing Subsystem Product Guide* (PG231)
13. *Versal AI Edge Gen 2 H.264/H.265/JPEG Video Codec Unit 2 Solutions LogiCORE IP Product Guide* (PG447)
14. *Vitis High-Level Synthesis User Guide* (UG1399)

Revision History

The following table shows the revision history for this document.

Section	Revision Summary
11/20/2025 Version 6.0	
Tile Format	Added the section.
Layer Width (0x0#18) Register	Description updated.
Layer Stride (0x0#20) Register	Description updated.

Section	Revision Summary
Layer Height (0x0#28) Register	Description updated.
General Settings	UltraRAM support added.
Prerequisites	Stride calculation updated.
05/29/2025 Version 5.3	
Y_U_V8	Added the section.
Y_U_V10	Added the section.
Y_U_V12	Added the section.
Layer Buffer Plane 1 (0x0#40), Layer Plane 2 Buffer (0x0#4C), and Layer Plane 3 Buffer (0x0#58) Registers	Updated the section.
User Parameters	Updated the table.
01/02/2024 Version 5.2	
IP Facts	Updated the section
N/A	Editorial changes
10/19/2022 Version 5.2	
Layer Width (0x0#18) Register	Updated the section
Chapter 4: Design Flow Steps	Added note for Line Buffer Width
04/27/2022 Version 5.2	
Top-Level Registers	Updated the section
CSC Coefficient Registers	Updated the section
Control (0x0000) Register	Updated the section
10/27/2021 Version 5.2	
N/A	Updated to support eight samples per clock.
08/06/2021 Version 5.2	
Table 16: Supported Platforms	Updated
Appendix B: Upgrading	Updated
02/04/2021 Version 5.1	
Chapter 6: Example Design	Updated with Vitis application flow for v5.1
06/10/2020 Version 5.0	
N/A	• Added CSC Coefficient Registers table. • Added Enable CSC Coefficient Registers parameter in the GUI. • Updated Figure 4-1. • Updated Table 4-1. • Updated Upgrading in the Vivado Design Suite section.
12/06/2019 Version 4.0	
N/A	• Updated for Example Design with Vitis application flow. • Added Layer 0 registers.
05/22/2019 Version 4.0	
N/A	Updated to support up to 16 overlay layers.
12/05/2018 Version 3.0	
N/A	Updated to show one main layer and eight overlay layers support.

Section	Revision Summary
04/04/2018 Version 3.0	
N/A	• Updated to support 8 overlay layers. • Added support for BGR8.
10/04/2017 Version 2.0	
N/A	• Added second buffer pointer for semi-planar formats. • Added 64-bit address support for memory mapped AXI4 interface. • Register map offsets re-ordered to handle both 32 and 64-bit addressing. • Added UYVY8 and BGRX8 memory formats. • Added per pixel alpha streaming formats RGBA and YUVA444.
04/05/2017 Version 1.0	
N/A	Added BGRA8, Y_UV10, Y_UV10_420, Y8, and Y10 to Memory Mapped AXI4 Interface.
10/05/2016 Version 1.0	
N/A	• Added YUV 4:2:0. • Updated Features in IP Facts. • Updated SDK directory link in IP Facts table. • Updated Feature Summary section. • Added RGBA8 to Y_UV8_420 sections. • Updated description for Layer Buffer (0x0048+i*8) Register section. • Added 0x4 0000 Logo Alpha Buffer table. • Updated description in Logo Red Buffer (0x1 0000) Register. • Updated description in General Design Guidelines section. • Updated Alpha Blending section. • Updated description to Layer Settings and Logo Layer Settings in Design Flow Steps chapter. • Added Enable Logo per Pixel Alpha in Vivado IDE Parameter to User Parameter Relationship table. • Updated Prerequisites section.
04/06/2016 Version 1.0	
N/A	Initial release.

Please Read: Important Legal Notices

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product releases, product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes. THIS INFORMATION IS PROVIDED "AS IS." AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

AUTOMOTIVE APPLICATIONS DISCLAIMER

AUTOMOTIVE PRODUCTS (IDENTIFIED AS "XA" IN THE PART NUMBER) ARE NOT WARRANTED FOR USE IN THE DEPLOYMENT OF AIRBAGS OR FOR USE IN APPLICATIONS THAT AFFECT CONTROL OF A VEHICLE ("SAFETY APPLICATION") UNLESS THERE IS A SAFETY CONCEPT OR REDUNDANCY FEATURE CONSISTENT WITH THE ISO 26262 AUTOMOTIVE SAFETY STANDARD ("SAFETY DESIGN"). CUSTOMER SHALL, PRIOR TO USING OR DISTRIBUTING ANY SYSTEMS THAT INCORPORATE PRODUCTS, THOROUGHLY TEST SUCH SYSTEMS FOR SAFETY PURPOSES. USE OF PRODUCTS IN A SAFETY APPLICATION WITHOUT A SAFETY DESIGN IS FULLY AT THE RISK OF CUSTOMER, SUBJECT ONLY TO APPLICABLE LAWS AND REGULATIONS GOVERNING LIMITATIONS ON PRODUCT LIABILITY.

Copyright